Practical Preimages for 19- and 20-Step Reduced SHA-256: Exact Algebraic Padding Inversion, Context Shaping, and Causal Bit-Slice Pruning

Author: Sebastian Wain **Date:** October 2026 **Repositories & Verification Suite:**

- 19-Step & 20-Step Padded Witnesses, Paper & Solvers: `https://github.com/srw/sha256-19step-preimages-with-padding`
- 20-Step Unpadded Compression Solver & Witness: `https://github.com/srw/sha256-20step-preimages-with-no-padding`
- Local Solver & Verification Suite: `solve20_padded_fast.c` (CPU), `solve20_padded_cuda.cu` / `solve20_padded_cuda_prod.cu` (CUDA GPU), `solve20_zero_a3.c` (Zero-a_3 Unpadded), `demos_20step.txt`, `examples_abc_hello_20step.txt`, `witnesses_20step.txt`, `FOUND_20ROUND_PREIMAGE.txt`, `verify_all_witnesses.py`

Abstract

We present the first practical preimage attacks on the **19-step** and **20-step reduced SHA-256 hash functions** respecting the standard initial value (IV), Davies–Meyer feed-forward, and full FIPS 180-4 Merkle–Damgård padding [1]. Whereas prior demonstrated preimages at 18–20 steps inverted only the unpadded compression function [6, 7, 9] and padding-aware meet-in-the-middle attacks at 24–47 steps remain strictly theoretical ($\geq 2^{240}$ evaluations) [2, 3, 4, 8], we construct valid 55-byte single-block and 119-byte two-block messages matching prescribed 256-bit digests. Building upon Basra's selector-saturation framework [9], we introduce: (1) an exact $O(1)$ algebraic inversion of the 72-bit FIPS 180-4 padding suffix via bit-disjoint pivot bits of $\Sigma_1(e_{13})$, achieving exact context shaping ($a_5 = a_6$); (2) a causal lower-triangular bit-slice equivalence theorem (**Carry-Sudoku**) that prunes 99.999% of right-hand contexts prior to evaluating $\Sigma_0(a_9)$; and (3) a 5-instruction closed-form $\mathbb{F}_2$ upper-half inverse paired with cache-tiled Q_0^{-1} tables (**Slot-Sudoku**) and 29-bit carry-tree pre-filtering. Our implementation requires 2^{32} precomputation evaluations (3.1 s) and achieves an online complexity of $2^{6.14}$ step-reduced evaluations per 19-step padded preimage ($0.47\ \mu$s on one NVIDIA L4 GPU) and $2^{38.14}$ evaluations per 20-step padded preimage (33.4 GPU-minutes on one L4 GPU, or 4.18 minutes across $8\times$ L4 GPUs). We also introduce a Zero-a_3 bi-directional bridge for the unpadded 20-step compression function (21.1 minutes on 16 CPU cores) and verify 29 concrete 20-step witnesses across standard public-label targets. These results concern step-reduced SHA-256 (19–20 of 64 steps) and do not threaten full 64-step SHA-256.

Keywords: SHA-256 · Preimage attack · Reduced-round cryptanalysis · FIPS 180-4 padding · Algebraic context shaping · Bit-slice pruning

---

1. Introduction

SHA-256 [1] hashes an L-bit message M by appending mandatory Merkle–Damgård strengthening (FIPS 180-4 §5.1.1)—a single `'1'` bit (`0x80` at a byte boundary), zero padding bits, and the 64-bit big-endian bit length L in the final two 32-bit words W_{14} and W_{15}—before applying a 64-step Davies–Meyer compression function initialized to the standard initial value IV.

In the cryptanalysis of step-reduced SHA-256, a persistent divide has separated **theoretical** attacks from **practical (demonstrated)** preimages:

- **Theoretical preimage and collision attacks:** Splice-and-cut meet-in-the-middle (MitM), partial-matching, biclique, and MILP-assisted attacks respect FIPS 180-4 padding from 24 steps [2] through 41–43 steps [3, 10], 45 steps [4], and 46–47 steps [8], with related differential and semi-free-start collision advances reaching 24–38 steps [11, 12, 13, 14, 15, 16]. Unlike SHA-1, where full-round collisions have been practically computed [18, 19], all 24–47-step SHA-256 preimage attacks require between 2^{240} and $2^{255.6}$ compression evaluations and remain strictly theoretical.
- **Practical (demonstrated) preimages:** Algebraic and SAT-based encodings produced padded preimages up to 16 steps [5, 20] and three unverified mailing-list messages at 17 steps [17]. More recently, SAT/Cube-and-Conquer [6, 7] and algebraic context shaping

[9] produced verified preimages at 18, 19, and 20 steps **exclusively for the unpadded compression function**, where all 16 message words $W_0, \ldots, W_{15}$ are unconstrained 32-bit degrees of freedom.

For a single-block 55-byte ($L = 440$-bit) padded message, FIPS 180-4 [1] fixes 72 bits at the end of the message block:

$$W_{15} = 440\ (\texttt{0x000001b8}), \qquad W_{14} = 0\ (\texttt{0x00000000}), \qquad W_{13} \bmod 256 = \texttt{0x80}. \tag{E1}$$

In backward-and-forward inversion of steps $0..18$ (19-step) or $0..19$ (20-step), W_{13}, W_{14}, W_{15} are tightly coupled with the message schedule expansion $W_{16}, \ldots, W_{19}$ and the internal state window $(a_5, \ldots, a_{12})$. Satisfying all 72 padding bits simultaneously with the selector-saturation and majority-absorption conditions [9] that decouple the left and right halves of the compression function is the central barrier between unpadded compression preimages and true padded hash preimages.

1.1 Upfront Demonstration of 19-Step and 20-Step Padded SHA-256 Preimages

To illustrate our results concretely before entering the algebraic derivations, below are four self-verifying **FIPS 180-4 padded preimages** for 20-step and 19-step SHA-256 ($\mathrm{SHA256}_{20}$ and $\mathrm{SHA256}_{19}$ with standard IV, Davies–Meyer feed-forward, and mandatory padding [1]; complete 29-witness suite in §8):

1. **20-Step Single-Block Padded Preimage for FIPS 180-4 Test Vector $H^* = \mathrm{SHA\text{-}256}(\text{“abc”})$:**

```
Target H*          = ba7816bf8f01cfea414140de5dae2223b00361a396177a9cb410ff61f20015ad
55-Byte Message M  = edf6e70a7001ae71b900f18119aaa1fdf6c1179b4268891262a9539be04d41236685e0c102e0ad23165e8b
Padded Block W0..7 = edf6e70a 7001ae71 b900f181 19aaa1fd f6c1179b 42688912 62a9539b e04d4123
Padded Block W8.15 = 6685e0c1 02e0ad23 165e8b71 57669819 302ddcb3 c6822680 00000000 000001b8
Verification       : SHA256_20(M) == H*  (W13 & 0xff = 0x80, W14 = 0, W15 = 440 bits)
```

2. **19-Step Single-Block Padded Preimage for $H^* = \mathrm{SHA\text{-}256}(\text{“abc”})$:**

```
55-Byte Message M  = 8ca254f8d04dd3fb04efaad6b1867588ffe833de49a144fedbf1839422892dc9f5a9158d8fc063290a14a9
Padded Block W8.15 = f5a9158d 8fc06329 0a14a9a6 c55dfa24 5642a024 7ec45a80 00000000 000001b8
Verification       : SHA256_19(M) == ba7816bf8f01cfea414140de5dae2223b00361a396177a9cb410ff61f20015ad
```

3. **20-Step Single-Block Padded Preimage for All-Zero Digest $H^* = 0000\ldots0000$:**

```
Target H*          = 0000000000000000000000000000000000000000000000000000000000000000
55-Byte Message M  = ec531306c85ad4a4ad211d2e1a03ece4d0ad4a0e6b4c2b9436b3babdcbd520597196f52c042b8cd95f1a39
Padded Block W8.15 = 7196f52c 042b8cd9 5f1a39f7 1b58b47b 0647c78e 65416f80 00000000 000001b8
Verification       : SHA256_20(M) == 0^256  (W13 & 0xff = 0x80, W14 = 0, W15 = 440 bits)
```

4. **20-Step & 19-Step Two-Block (119-Byte, Plain-ASCII Block 1) Padded Preimages for $H^* = \mathrm{SHA\text{-}256}(\text{“”})$:**

```
Target H*          = e3b0c44298fc1c149afbf4c8996fb92427ae41e4649b934ca495991b7852b855
Block 1 (64B ASCII)= "This first block is plain text; the second one was computed....."
20-Step Block 2 M2 = df28e21bb6b61202ba87baa72096ee0a3ac325cf1f12056e7cfe2ae1735b4d106b8f6e0b22eb9b504ed936
19-Step Block 2 M2 = fe409cc930128a720481f86c0503d7a018519a129df449139720bb5eef520895f8b515685bb404817be301
Padding Suffix     : ...80 00000000 000003b8 (L = 952 bits = 119 bytes; full Merkle-Damgard 2-block chainin
```

Table 1.1: Comparison of theoretical and practical (demonstrated) preimage attacks on step-reduced SHA-256.

Work & Reference	Steps	Attacked Object	FIPS 180-4 Padding?	Target Type	Complexity / Runtime	Demonstrated Witness?
Isobe & Shibutani [2]	24	Hash (1 block)	Yes	Arbitrary	2^{240} evaluations	No (theoretical)
Sasaki & Aoki [10]	41	Hash	Yes	Arbitrary	$2^{253.5}$ evaluations	No (theoretical)

Work & Reference	Steps	Attacked Object	FIPS 180-4 Padding?	Target Type	Complexity / Runtime	Demonstrated Witness?
Aoki et al. [3]	43	Hash	Yes	Arbitrary	$2^{254.9}$ evaluations	No (theoretical)
Khovratovich et al. [4]	45 / 52	Hash / Compression	Yes / No	Arbitrary	$2^{255.5}/2^{255.0}$ evaluations	No (theoretical)
Guo et al. [8]	46 / 47	Hash	Yes	Arbitrary	$2^{254.5}/2^{255.6}$ evaluations	No (theoretical)
Homsirikamol et al. [5]	16	Hash (446-bit)	Yes	Fixed	~ 192 core-h (SAT)	No (not printed)
McDair [17]	17	Hash (31B / 55B)	Yes	1 challenge	Unspecified (no method)	Yes (3 messages)
Davydov et al. [6]	17 / 18	Compression	No	All-zero (`00..00`)	Seconds (SAT)	Yes (unpadded only)
Zaikin [7]	19	Compression	No	All-ones (`ff..ff`)	18.5 h on 192 cores (SAT)	Yes (unpadded only)
Basra [9]	19 / 20	Compression	No	Arbitrary / `ff..ff`	0.01 s (19) / 2.45–16.2 GPU-h (20)	Yes (unpadded only)
This Work (`solve20_zero_a3.c`)	**20**	**Compression**	**No**	**`deadbeef...` + 12 Labels**	**$2^{36.85}$ evals (`21.1 min` 16 CPU cores)**	**Yes (13 witnesses, §8.4)**
This Work (`solve20_padded_cuda.cu`)	**19**	**Full Hash (55B / 119B)**	**Yes [1]**	**Public-label digests**	**$2^{6.14}$ evals (`0.47 µs` 1× L4 GPU)**	**Yes (37+ witnesses, §8.2–§8.4)**
This Work (`solve20_padded_cuda_prod.cu`)	**20**	**Full Hash (55B / 119B)**	**Yes [1]**	**`00..00` + 14 Labels + `""`**	**$2^{38.14}$ evals (`33.4 min` 1× L4 / `4.18 min` 8× L4)**	**Yes (16 witnesses, §8.1–§8.4)**

1.2 Technical Contributions and Organization

1. **Exact $O(1)$ Algebraic Inversion of FIPS 180-4 Padding (§3–§4, `[E15]` – `[E22]`):** Extending Basra's selector-saturation framework [9], we exploit the bit-disjointness of $\Sigma_1(e_{13}) \bmod 256$ with respect to bits $18..31$ of e_{13} to invert $(W_{13} \bmod 256) = \texttt{0x80}$ in $O(1)$ time with 100% inner-loop validity. Combined with Ch-annihilation ($e_{15} = \texttt{0xffffffff}$, $a_{10} = a_{11}$), this yields exact $D = a_5 \oplus a_6 = 0$ padded contexts without soft-shaping survival loss.
2. **Causal Bit-Slice Right-Context Pruning ("Carry-Sudoku", §5, `[E23]` – `[E26]`):** We prove **Theorem 1** (`[E23]`): $a_5 = a_6$ decomposes into a lower-triangular carry condition $((S \oplus X) \wedge D) = 0$ that is completely independent of a_6 and $\Sigma_0(a_9)$, and a complementary condition $((a_6 \oplus S) \wedge \neg D) = 0$. Pruning e_{13} across 7 causal bit-slices plus rotation-invariant windows of $\Sigma_0(a_9)$ eliminates 99.999% of branches early, generating $114{,}507$ exact $D = 0$ padded contexts/s on one NVIDIA L4 GPU.
3. **Closed-Form $\mathbb{F}_2$ Reconstruction, Cache-Tiled Q_0^{-1} Tables ("Slot-Sudoku"), and Carry-Tree Pre-Filtering (§6, `[E27]` – `[E37]` , `[E42]` – `[E45]`):** We prove **Theorem 2** (`[E28]` – `[E29]`), recovering the upper 16 bits of $u \in Q_0^{-1}(y)$ in 5 bitwise instructions with zero lookup tables. Pairing this with 29-bit carry-tree pre-filtering, L2 cache-tile partitioning, and Octet-Hexadecet (T_{16}, T_{17}) coalescing reduces online work to $70.58 \approx 2^{6.14}$ step-reduced SHA-256 evaluations per 19-step padded solution (2.14×10^6 sol19/s on one L4 GPU).
4. **Zero-a_3 Bi-Directional Bridge, Complete 19- and 20-Step Witness Suites, and Structural Complexity Bounds (§7–§9, `[E38]` – `[E41]`):** We present a Zero-a_3 bridge for unpadded 20-step compression (§7), provide the complete, un-truncated hex witnesses for all 19-step and 20-step single-block padded, two-block chosen-ASCII-prefix padded, and unpadded benchmarks (§8), and establish a 15-Theory Structural Barrier Theorem characterizing the exact degree-of-freedom conservation laws at 19, 20, and 21 steps (§9.2).

---

2. Preliminaries and the Scalar a-Sequence Formulation

All arithmetic is modulo 2^{32}. Denote 32-bit right rotation by $\mathrm{ROR}(x, n)$ and logical right shift by $\mathrm{SHR}(x, n)$. The SHA-256 boolean and $\mathbb{F}_2$-linear functions [1] are:

$$\begin{aligned} \mathrm{Ch}(e,f,g) &= (e \wedge f) \oplus (\neg e \wedge g), & \mathrm{Maj}(a,b,c) &= (a \wedge b) \oplus (a \wedge c) \oplus (b \wedge c), \\ \Sigma_0(x) &= \mathrm{ROR}(x,2) \oplus \mathrm{ROR}(x,13) \oplus \mathrm{ROR}(x,22), & \Sigma_1(x) &= \mathrm{ROR}(x,6) \oplus \mathrm{ROR}(x,11) \oplus \mathrm{ROR}(x,25), \\ \sigma_0(x) &= \mathrm{ROR}(x,7) \oplus \mathrm{ROR}(x,18) \oplus \mathrm{SHR}(x,3), & \sigma_1(x) &= \mathrm{ROR}(x,17) \oplus \mathrm{ROR}(x,19) \oplus \mathrm{SHR}(x,10). \end{aligned} \tag{E2}$$

For step indices $t \geq 16$, the message schedule expands $W_0, \ldots, W_{15}$ via:

$$W_t = \sigma_1(W_{t-2}) + W_{t-7} + \sigma_0(W_{t-15}) + W_{t-16}. \tag{E3}$$

Following [9], we define the coupled schedule operator $Q_0 : \mathbb{Z}_{2^{32}} \to \mathbb{Z}_{2^{32}}$ by:

$$Q_0(u) = \sigma_0(u) - u \pmod{2^{32}}. \tag{E4}$$

2.1 Reduction to the Scalar a-Sequence (`[E5]` – `[E7]`)

Let $(a_t, b_t, c_t, d_t, e_t, f_t, g_t, h_t)$ be the 8 working registers after step $t-1$ (with $t=0$ denoting IV). Because each step shifts $b \leftarrow a, c \leftarrow b, d \leftarrow c$ and $f \leftarrow e, g \leftarrow f, h \leftarrow g$, the entire 8-register trajectory across R steps is uniquely parameterized by a single scalar sequence $a_{-7}, a_{-6}, \ldots, a_R$ [3, 9]:

- **Initial Boundary ($s \in \{-7, \ldots, 0\}$):** Fixed uniquely by IV (`window_from_state`):

$$(a_0, a_{-1}, a_{-2}, a_{-3}) = (\mathrm{IV}_0, \mathrm{IV}_1, \mathrm{IV}_2, \mathrm{IV}_3), \qquad (e_0, e_{-1}, e_{-2}, e_{-3}) = (\mathrm{IV}_4, \mathrm{IV}_5, \mathrm{IV}_6, \mathrm{IV}_7). \tag{E5}$$

- **Terminal Boundary ($s \in \{R-7, \ldots, R\}$):** For an R-step target digest $H^* = (H_0^*, \ldots, H_7^*)$, Davies–Meyer feed-forward fixes the terminal state $st_i = H_i^* - \mathrm{IV}_i$, which uniquely determines the 8 terminal words $a_{R-7}, \ldots, a_R$:

$$\begin{aligned} &\text{For } R = 19: \quad a_{12}, a_{13}, \ldots, a_{19} \text{ are fixed by } H^* - \mathrm{IV}, \\ &\text{For } R = 20: \quad a_{13}, a_{14}, \ldots, a_{20} \text{ are fixed by } H^* - \mathrm{IV}. \end{aligned} \tag{E6}$$

- **Internal State & Message Recovery:** For every step index s and $t \in \{0, \ldots, R-1\}$, the e-register e_s and message word W_t are exact functions of five consecutive a-words (`E_arr` and `Wt_arr`):

$$\begin{aligned} e_s &= a_s - \Sigma_0(a_{s-1}) - \mathrm{Maj}(a_{s-1}, a_{s-2}, a_{s-3}) + a_{s-4}, \\ W_t &= e_{t+1} - a_{t-3} - e_{t-3} - \Sigma_1(e_t) - \mathrm{Ch}(e_t, e_{t-1}, e_{t-2}) - K_t. \end{aligned} \tag{E7}$$

Thus, for $R = 20$ (resp. $R = 19$), the only free internal state variables between the IV boundary and the target boundary are the **12 words** $a_1, a_2, \ldots, a_{12}$ (resp. **11 words** $a_1, \ldots, a_{11}$). Any assignment to these words automatically hashes to H^* after R steps if and only if the recovered message words $W_0, \ldots, W_{R-1}$ from `[E7]` satisfy the message schedule equations `[E3]` for $t = 16, \ldots, R-1$ and the FIPS 180-4 padding constraints `[E1]` .

---

3. Selector Saturation and Exact Context Shaping ($D = 0$)

In general, $W_{16}, W_{17}, W_{18}, W_{19}$ depend on the Left variables (a_1, a_2, a_3, a_4) both through the early message words $(W_0, W_1, W_2, W_3, W_4)$ on the right-hand side of `[E3]` and through $(W_9, W_{10}, W_{11}, W_{12}, W_{14}, W_{15})$ via the state words e_5, e_6, e_7, e_8.

Following Basra's selector-saturation principle [9], we pin the two middle selector registers:

$$e_9 = \texttt{0xffffffff}, \qquad e_{10} = \texttt{0xffffffff}. \tag{E8}$$

Because $\mathrm{Ch}(\texttt{0xffffffff}, y, z) = y$, setting $e_9 = e_{10} = \texttt{0xffffffff}$ blocks (e_7, e_8) inside $\mathrm{Ch}(e_9, e_8, e_7) = e_8$, $\mathrm{Ch}(e_{10}, e_9, e_8) = \texttt{0xffffffff}$, and $\mathrm{Ch}(e_{11}, e_{10}, e_9) = \texttt{0xffffffff}$. Expanding `[E7]` at $s = 9, 10$ expresses a_5 and a_6 purely in terms of the Right words (a_7, a_8, a_9, a_{10}):

$$\begin{aligned} a_5 &= \texttt{0xffffffff} - a_9 + \Sigma_0(a_8) + \mathrm{Maj}(a_8, a_7, a_6), \\ a_6 &= \texttt{0xffffffff} - a_{10} + \Sigma_0(a_9) + \mathrm{Maj}(a_9, a_8, a_7). \end{aligned} \tag{E9}$$

3.1 Exact Shaping ($D = a_5 \oplus a_6 = 0$) (`[E10]` – `[E14]`)

Let $D_{\text{shape}} = a_5 \oplus a_6$. Under `[E8]`, the dependence of $W_9, W_{10}, W_{11}, W_{12}$ on the Left variables (a_1, a_2, a_3, a_4) collapses to [9]:

$$\begin{aligned} W_9 &= c_9 - a_1, \\ W_{10} &= c_{10} - a_2, \\ W_{11} &= c_{11} - a_3 - \big(\mathrm{Maj}(a_6, a_5, a_4) - (a_6 \wedge a_5)\big), \\ W_{12} &= \mathrm{const}(\mathrm{Right}) + a_4 - \mathrm{Maj}(a_5, a_4, a_3), \end{aligned} \tag{E10}$$

where c_9, c_{10}, c_{11} depend **only** on the Right context $(a_5, \ldots, a_{12})$.

When we enforce **exact context shaping**:

$$D_{\text{shape}} = a_5 \oplus a_6 = 0 \iff a_5 = a_6, \tag{E11}$$

the majority term in W_{11} becomes identically constant for **every** $a_4 \in \mathbb{Z}_{2^{32}}$:

$$\mathrm{Maj}(a_6, a_5, a_4) = \mathrm{Maj}(a_5, a_5, a_4) = a_5 = a_6 \wedge a_5 \implies W_{11} = c_{11} - a_3 \quad (\text{exact, zero } a_4 \text{ leakage}). \tag{E12}$$

(In our first-generation 19-step solver `src/pre19.c`, soft shaping $\mathrm{popcount}(a_5 \oplus a_6) \le 3$ leaked a_4 into W_{11} on up to 3 bits and reduced chain survival by $2^{\mathrm{popcount}(a_5 \oplus a_6)} = 2 \times$ –8×. Exact shaping `[E11]` eliminates this penalty completely.)

Simultaneously, the remaining nonlinear condition in W_{12} (`[E10]`):

$$\mathrm{Maj}(a_5, a_4, a_3) = a_4 \iff \big((a_4 \oplus a_5) \wedge M_3\big) = 0, \qquad M_3 := \neg(a_3 \oplus a_5), \tag{E13}$$

causes $+a_4$ inside e_8 and $-\mathrm{Maj}(a_5, a_4, a_3) = -a_4$ inside e_7 to cancel identically in W_{12}. Consequently, whenever `[E11]` and `[E13]` hold, $W_{12}, W_{13}, \ldots, W_{19}$ are **100% independent of** (a_1, a_2, a_3, a_4). Table 3.1 summarizes the resulting bit-level state conditions across all 20 steps in the format of [3, 8, 11].

Table 3.1: Bit-level state conditions, neutral/coupled words, and schedule constraints across steps $s \in \{-7, \ldots, 20\}$.

Step Range s	Register a_s Condition	Register e_s Condition	Message Word W_{s-1} Role & Constraint	Governing Equation
$-7 \ldots 0$	Fixed by $\mathrm{IV}_{0..3}$	Fixed by $\mathrm{IV}_{4..7}$	— (Initial boundary)	`[E5]`
1	Free root $a_1 \in \mathbb{Z}_{2^{32}}$	$e_1(a_1)$	$W_0 = a_1 + C_{W_0}$ (Carry-tree pre-filtered into L2 tile)	`[E7]` , `[E42]`
$2, 3$	$a_2 = W_1 + G_1$, $a_3 = W_2 + G_2$	$e_2(a_{1..2})$, $e_3(a_{1..3})$	$W_1 \in Q_0^{-1}(T_{16} + F_1)$, $W_2 \in Q_0^{-1}(T_{17} + G_2 - W_1)$; $\mathrm{wt}(M_3) \le 14$	`[E14]` , `[E34]`
4	$\big((a_4 \oplus a_5) \wedge \neg(a_3 \oplus a_5)\big) = 0$	$e_4(a_{1..4})$	$W_3 \in Q_0^{-1}(T_{18} + G_3 - W_2)$ ($\mathrm{Maj}(a_5, a_4, a_3) = a_4$)	`[E13]` , `[E14]`
$5, 6$	**Exact Shaping:** $a_5 = a_6$ ($D = 0$)	e_5, e_6 (coupled)	W_4 tested at step 20 via $\sigma_0(W_4) + W_3 = T_{19}$; W_5 free	`[E11]` , `[E36]`
$7, 8$	Closed-form $a_7(e_{13})$, $a_8 = \mathrm{const}$	e_7, e_8 (absorbed)	W_6, W_7 free; $+a_4$ in e_8 cancels $-\mathrm{Maj}$ in e_7	`[E17]` , `[E22]`

Step Range s	Register a_s Condition	Register e_s Condition	Message Word W_{s-1} Role & Constraint	Governing Equation
$9, 10$	$a_9 = e_{13} - c_{e_{13}}$, $a_{10} = a_{11}$	$e_9 = e_{10} =$ `0xffffffff`	W_8 free; $W_9 = c_9 - a_1$ (linear in a_1)	`[E8]`, `[E10]`
$11, 12$	a_{11} fixed by e_{15}, a_{12} seed	$e_{11}(e_{13})$, $e_{12} = \text{const}$	$W_{10} = c_{10} - a_2$, $W_{11} = c_{11} - a_3$ (zero a_4 leakage)	`[E12]`, `[E16]`
$13, 14$	a_{13}, a_{14} fixed by $H^* - \text{IV}$	e_{13} (7-stage causal), e_{14}	$W_{12} = \text{const}(\text{Right})$; **Padding:** $W_{13} \bmod 256 =$ `0x80`	`[E10]`, `[E21]`
$15, 16$	a_{15}, a_{16} fixed by $H^* - \text{IV}$	$e_{15} =$ `0xffffffff`, e_{16}	**Padding:** $W_{14} =$ `0x00000000`, $W_{15} =$ `0x000001b8` (440)	`[E1]`, `[E22]`
$17 \dots 20$	$a_{17..20}$ fixed by $H^* - \text{IV}$	$e_{17..20}$ fixed	Schedule words $W_{16..19}$ define $(T_{16}, T_{17}, T_{18}, T_{19})$	`[E6]`, `[E14]`

Substituting $W_0 = a_1 + C_{W_0}$ and $a_{t+1} = W_t + G_t(a_1, \dots, a_t)$ into the message schedule equations `[E3]` for $t = 16, 17, 18$ rewrites them as three chained Q_0-inversions:

$$\begin{aligned}
&\text{Level 1 } (t=16): \quad Q_0(W_1) = T_{16} + F_1(a_1), && T_{16} := W_{16} - \sigma_1(W_{14}) - c_9, \\
&\text{Level 2 } (t=17): \quad Q_0(W_2) = T_{17} + G_2(a_1, a_2) - W_1, && T_{17} := W_{17} - \sigma_1(W_{15}) - c_{10}, \\
&\text{Level 3 } (t=18): \quad Q_0(W_3) = T_{18} + G_3(a_1, a_2, a_3) - W_2, && T_{18} := W_{18} - \sigma_1(W_{16}) - c_{11}.
\end{aligned} \tag{E14}$$

---

4. Exact $O(1)$ Algebraic Inversion of FIPS 180-4 Padding (`W13..W15`)

To construct a valid 55-byte single-block padded message (`[E1]`), the Right variables $(a_5, \dots, a_{12})$ must simultaneously satisfy $W_{15} = 440$, $W_{14} = 0$, $(W_{13} \bmod 256) =$ `0x80`, $e_9 = e_{10} =$ `0xffffffff` (`[E8]`), and $a_5 = a_6$ (`[E11]`).

4.1 Closed-Form Absorption of $W_{15} = 440$ and $W_{14} = 0$ (`[E15]`–`[E17]`)

Given (a_{10}, a_{11}, a_{12}) (for $R = 20$, where $a_{13..20}$ are fixed by H^*), the registers e_{16}, e_{15}, e_{14} are determined via `[E7]`:

$$\begin{aligned}
e_{16} &= a_{16} - \Sigma_0(a_{15}) - \text{Maj}(a_{15}, a_{14}, a_{13}) + a_{12}, \\
e_{15} &= a_{15} - \Sigma_0(a_{14}) - \text{Maj}(a_{14}, a_{13}, a_{12}) + a_{11}, \\
e_{14} &= a_{14} - \Sigma_0(a_{13}) - \text{Maj}(a_{13}, a_{12}, a_{11}) + a_{10}.
\end{aligned} \tag{E15}$$

Expanding $W_{15} = 440$ and $W_{14} = 0$ via `[E7]` isolates e_{12} and e_{11} as closed-form functions of a single 32-bit word e_{13} (equivalently $a_9 = e_{13} - c_{e_{13}}$):

$$\begin{aligned}
e_{12} &= \underbrace{(e_{16} - a_{12} - \Sigma_1(e_{15}) - K_{15} - 440)}_{c_{e_{12}}} - \text{Ch}(e_{15}, e_{14}, e_{13}), \\
e_{11} &= \underbrace{(e_{15} - a_{11} - \Sigma_1(e_{14}) - K_{14})}_{c_{e_{11}}} - \text{Ch}(e_{14}, e_{13}, e_{12}).
\end{aligned} \tag{E16}$$

Then a_8 and a_7 are recovered in closed form:

$$a_8 = e_{12} + \Sigma_0(a_{11}) - a_{12} + \text{Maj}(a_{11}, a_{10}, a_9), \qquad a_7 = e_{11} + \Sigma_0(a_{10}) - a_{11} + \text{Maj}(a_{10}, a_9, a_8). \tag{E17}$$

Thus **every** choice of $(a_{12}, a_{11}, a_{10}, e_{13})$ satisfies $W_{15} = 440$ and $W_{14} = 0$ identically.

4.2 Exact $O(1)$ Pivot Inversion of $(W_{13} \bmod 256) =$ `0x80` (`[E18]`–`[E22]`)

Since $e_{10} =$ `0xffffffff`, the formula for W_{13} (`[E7]`) is:

$$W_{13} = \underbrace{(e_{14} - a_{10} - \texttt{0xffffffff} - K_{13})}_{c_{W_{13}}} - \Sigma_1(e_{13}) - \mathrm{Ch}(e_{13}, e_{12}, e_{11}). \tag{E18}$$

Reducing `[E18]` modulo 256:

1. Because Ch is bitwise and modular subtraction propagates carries only from lower bits to higher bits, the low 8 bits $e_{12,\mathrm{lo}} := e_{12} \bmod 256$ and $e_{11,\mathrm{lo}} := e_{11} \bmod 256$ in `[E16]` depend **only on the low 8 bits** $e_{13,\mathrm{lo}} := e_{13} \bmod 256$.
2. Once $e_{13,\mathrm{lo}} \in [0, 255]$ is chosen, the required low byte of $\Sigma_1(e_{13})$ is fixed:

$$R_{\mathrm{lo}} := \left(c_{W_{13}} - \mathrm{Ch}(e_{13,\mathrm{lo}}, e_{12,\mathrm{lo}}, e_{11,\mathrm{lo}}) - \texttt{0x80}\right) \bmod 256. \tag{E19}$$

3. Inspecting bit $b \in \{0, \ldots, 7\}$ of $\Sigma_1(e_{13}) = \mathrm{ROR}(e_{13}, 6) \oplus \mathrm{ROR}(e_{13}, 11) \oplus \mathrm{ROR}(e_{13}, 25)$:

$$\left(\Sigma_1(e_{13})\right)_b = (e_{13})_{b+6} \oplus (e_{13})_{b+11} \oplus (e_{13})_{(b+25) \bmod 32}. \tag{E20}$$

- For $b \in \{0, \ldots, 6\}$, $(b + 25) \bmod 32 = b + 25 \in \{25, \ldots, 31\}$, which appears **nowhere else** in `[E20]`.
- For $b = 7$, $(7 + 25) \bmod 32 = 0 \in \{0, \ldots, 7\}$, while $(e_{13})_{7+11} = (e_{13})_{18}$ appears **nowhere else** in `[E20]`.
- Furthermore, bits $19..24$ of e_{13} **do not appear anywhere in `[E20]`** and **do not affect** $e_{13,\mathrm{lo}}$.

Therefore, for any assignment to bits $0..17$ of e_{13} ($e_{13,\mathrm{base}}$), the 8 pivot bits (`bit 18` and `bits 25..31`) are uniquely determined in closed form:

$$\Delta := \left(R_{\mathrm{lo}} \oplus \Sigma_1(e_{13,\mathrm{base}})\right) \bmod 256, \qquad e_{13,\mathrm{fixed}} := e_{13,\mathrm{base}} \mid \left((\Delta \wedge \texttt{0x7f}) \ll 25\right) \mid \left(((\Delta \gg 7) \wedge 1) \ll 18\right) \tag{E21}$$

while bits $19..24$ remain 6 completely free degrees of freedom where 100% **of assignments satisfy** $(W_{13} \bmod 256) = \texttt{0x80}$.

Finally, in `solve20_padded_fast.c` and `solve20_padded_cuda.cu`, we apply Ch**-annihilation** by pinning $e_{15} = \texttt{0xffffffff}$ (fixing a_{11} from a_{12} via `[E15]`) and setting $a_{10} = a_{11}$. This simultaneously forces $\mathrm{Ch}(e_{15}, e_{14}, e_{13}) = e_{14}$ (**making e_{12} and a_8 100% constant across all 2^{24} choices of e_{13}**), simplifies $\mathrm{Maj}(a_{11}, a_{10}, a_9) = a_{11}$, and linearizes $\mathrm{Maj}(a_{10}, a_9, a_8) = (a_{10} \wedge a_8) + ((a_{10} \oplus a_8) \wedge a_9)$:

$$e_{15} = \texttt{0xffffffff}, \quad a_{10} = a_{11} \implies e_{12} = c_{e_{12}} - e_{14} = \mathrm{const}, \quad a_8 = e_{12} + \Sigma_0(a_{11}) - a_{12} + a_{11} = \mathrm{const} \tag{E22}$$

---

5. Carry-Sudoku: 7-Stage Causal Bit-Slice & Rotation-Invariant Right-Context Search

Even with `[E21]` – `[E22]`, each a_{12} defines 2^{24} padded candidates for e_{13}, and testing $a_5 = a_6$ (`[E11]`) naively requires evaluating $\Sigma_0(a_9)$ and a_6 across those candidates. We now prove that causal lower-triangular bit-slice pruning—which we term **Carry-Sudoku**—eliminates 99.999% **of all e_{13} branches** *before* $\Sigma_0(a_9)$ or a_6 is ever computed.

5.1 The Exact Equivalence Theorem (`[E23]`)

Theorem 1 (Carry-Sudoku Exact Equivalence). *Under `[E9]`, define* $X := \texttt{0xffffffff} - a_9 + \Sigma_0(a_8)$, $D := a_8 \oplus a_7$, *and* $S := X + \mathrm{Ch}(D, X, a_8) \pmod{2^{32}}$. *Then:*

$$a_5 = a_6 \iff \Big(((S \oplus X) \wedge D) = 0\Big) \ \wedge \ \Big(((a_6 \oplus S) \wedge \neg D) = 0\Big). \tag{E23}$$

Proof. By `[E9]`, $a_5 = X + \mathrm{Maj}(a_8, a_7, a_6)$. Decompose $\mathrm{Maj}(a_8, a_7, a_6)$ bitwise with respect to $D = a_8 \oplus a_7$:

- On bit positions where $D_b = 0$ ($a_{8,b} = a_{7,b}$), $\mathrm{Maj}(a_8, a_7, a_6)_b = a_{8,b}$ is independent of a_6.
- On bit positions where $D_b = 1$ ($a_{8,b} \neq a_{7,b}$), $\mathrm{Maj}(a_8, a_7, a_6)_b = a_{6,b}$.

Hence $\mathrm{Maj}(a_8, a_7, a_6) = (D \wedge a_6) \oplus (\neg D \wedge a_8) = \mathrm{Ch}(D, a_6, a_8)$. Now compare $a_5 = X + \mathrm{Ch}(D, a_6, a_8)$ with $S = X + \mathrm{Ch}(D, X, a_8)$ by induction from bit $b = 0$ to 31:

- Suppose $a_5 = a_6$ on bits $0..b-1$, so the incoming carry c_b into bit b of $X + \mathrm{Ch}(D, a_6, a_8)$ equals the incoming carry into bit b of $S = X + \mathrm{Ch}(D, X, a_8)$:
- If $D_b = 1$, then $\mathrm{Ch}(D, a_6, a_8)_b = a_{6,b}$, so $a_{5,b} = X_b \oplus a_{6,b} \oplus c_b$. Thus $a_{5,b} = a_{6,b} \iff X_b \oplus c_b = 0$, and when $X_b \oplus c_b = 0$, the outgoing carry from bit b is $\mathrm{Maj}(X_b, a_{6,b}, c_b) = X_b = \mathrm{Maj}(X_b, X_b, c_b)$, which is **identical** to the outgoing carry of $S = X + \mathrm{Ch}(D, X, a_8)$ where $S_b = X_b \oplus X_b \oplus c_b = c_b = X_b$, i.e. $(S_b \oplus X_b) = 0$.
- If $D_b = 0$, then $\mathrm{Ch}(D, a_6, a_8)_b = a_{8,b} = \mathrm{Ch}(D, X, a_8)_b$, so $a_{5,b} = S_b$ and the outgoing carry is again identical to that of S. Thus $a_{5,b} = a_{6,b} \iff a_{6,b} = S_b$. $\square$

5.2 7-Stage Causal Bit-Slice Pruning & Rotation-Invariant $\Sigma_0(a_9)$ Windows (`[E24]` – `[E26]`)

Theorem 1 has a decisive algorithmic consequence: the first condition in `[E23]`,

$$\big((S \oplus X) \wedge D\big) = 0, \qquad S = X + \mathrm{Ch}(D, X, a_8), \tag{E24}$$

does not depend on a_6 or $\Sigma_0(a_9)$ at all, and because $a_9 = e_{13} - c_{e_{13}}$, $X = c_{a_5} - a_9$, and $a_7 = c_{a7,\mathrm{full}} - (e_{14} \wedge e_{13}) + \big((a_{10} \oplus a_8) \wedge a_9\big)$ involve only bitwise operations and modular additions/subtractions, **the lowest k bits of X, a_7, D, and S depend exclusively on the lowest k bits of e_{13}**.

Consequently, `ctx_gen_worker` (CPU) and `ctx_gen_cuda_kernel` (GPU) assign e_{13} from LSB to MSB in **7 causal stages** (formalized in **Algorithm 1**):

$$\begin{aligned}
\text{Stage 1a (bits 0..3):}\quad & \big((S \oplus X) \wedge D \wedge \texttt{0xf}\big) = 0, \\
\text{Stage 1b (bits 4..7):}\quad & \big((S \oplus X) \wedge D \wedge \texttt{0xff}\big) = 0 \implies \text{fixes } R_{\mathrm{lo}} \text{ and } (e_{13})_{18} \oplus (e_{13})_{13}, \\
\text{Stage 1c (bits 8..11):}\quad & \big((S \oplus X) \wedge D \wedge \texttt{0xfff}\big) = 0, \\
\text{Stage 1d (bits 12..14 + exact bit 18):}\quad & \big((S \oplus X) \wedge D \wedge \texttt{0x7fff}\big) = 0, \\
\text{Stage 1e (bits 15..17, checked to bit 18):}\quad & \big((S \oplus X) \wedge D \wedge \texttt{0x7ffff}\big) = 0 \implies \text{fixes bits 25..31 via } [\mathrm{E21}].
\end{aligned} \tag{E25}$$

In Stage 1d, once bit 13 of e_{13} is chosen, bit 18 of e_{13} is immediately determined by $(R_{\mathrm{lo}} \gg 7) \oplus (e_{13})_0 \oplus (e_{13})_{13}$ (`[E20]`), allowing Stage 1e to test a **19-bit Carry-Sudoku mask (`0x7ffff`)** on 18 chosen bits *before* $\Sigma_1(e_{13,\mathrm{base}})$ is evaluated.

Next, once bits $0..18$ and $25..31$ are fixed (26 of 32 bits of e_{13}), only bits $19..24$ (`s19` and `s22`) remain free. Under $\Sigma_0(a_9) = \mathrm{ROR}(a_9, 2) \oplus \mathrm{ROR}(a_9, 13) \oplus \mathrm{ROR}(a_9, 22)$, bits $19..24$ of a_9 (allowing 1 carry bit into bit 25) land exclusively in bit ranges `[17..23]`, `[6..12]`, and `[29..31, 0..2]`. Consequently, **bits `13..15` and `3..4` of $\Sigma_0(a_9)$ are 100% rotation-invariant across all 64 values of bits $19..24$**. Since S and D are already fixed through bit 18, we check the exact equivalence `a6_0 == a5_0` on those two rotation-invariant slices (`Stage 2`) before looping over `s19` (`Stage 3a`, mask `0x3fffff`) and `s22` (`Stage 3b`, full 32-bit `[E23]`):

$$\big(((a_{6,0} - a_{5,0}) \gg 13) + 1\big) \wedge \texttt{0xe} = 0 \quad \wedge \quad \big(((a_{6,0} - a_{5,0}) \gg 3) + 1\big) \wedge \texttt{0x6} = 0. \tag{E26}$$

```
Algorithm 1: Causal 7-Stage Carry-Sudoku Right-Context Generator
Input: Target terminal words a_{13}..a_{20} = H* - IV, seed range a_{12} in [0, 2^32)
Output: Reservoir R of exact D = (a_5 ^ a_6) == 0 FIPS 180-4 padded contexts
1.  for each seed a_{12} in parallel do
2.    Set e_{15} = 0xffffffff; compute a_{11} via [E15] and set a_{10} = a_{11}            // [E22] Ch-annihilatio
3.    Compute constants e_{16}, e_{14}, e_{12} = c_{e12} - e_{14}, and a_8 = const         // [E16]-[E17]
4.    Sweep bits 0..3 and 4..7 of e_{13} filtering ((S ^ X) & D & mask) == 0 (masks 0xf, 0xff)
5.    Compute low-byte padding target R_lo = (c_{W13} - Ch(e_{13}, e_{12}, e_{11}) - 0x80) mod 256  // [E19]
6.    Sweep bits 8..11, 12..14 (fixing bit 18 via [E20]), and 15..17 filtering masks 0xfff, 0x7fff, 0x7ffff
7.    Fix bits 25..31 of e_{13} in O(1) via Delta = (R_lo ^ Sigma_1(e_{13,base})) mod 256   // [E21]
8.    Test rotation-invariant Sigma_0(a_9) windows on bits 13..15 and 3..4                  // [E26] Stage 2
9.    for s_{19} in 0..7 (mask 0x3fffff) and s_{22} in 0..7 do                              // Stage 3a, 3b
10.     if ((S ^ X) & D) == 0 and ((a_6 ^ S) & ~D) == 0 then                                // Theorem 1 [E23]
11.       Append context (a_5..a_{12}, T_{16}, T_{17}, T_{18}, T_{19}) to Reservoir R
```

On GPU (`generate_right_ctx_gpu_kernel`), mapping Algorithm 1 onto warp-cooperative lockstep stages generates $114{,}507$ **exact $D = 0$ padded contexts per second** on 1× NVIDIA L4 GPU ($1{,}048{,}576$ contexts in 9.16 seconds).

---

6. Slot-Sudoku & Zero-Table Closed-Form $\mathbb{F}_2$ Inverse for Left-to-Right Q_0^{-1} Chains

With an exact $D = a_5 \oplus a_6 = 0$ padded Right context in hand, we solve the three chained Q_0^{-1} equations `[E14]` from Left to Right.

6.1 Closed-Form 5-Instruction $\mathbb{F}_2$ Inverse (`reconstruct_u` , `[E27]` – `[E29]`)

Storing full 32-bit preimages u of $y = Q_0(u) = \sigma_0(u) - u$ requires 4 bytes per slot. Splitting $u = (u_{\text{hi}} \ll 16) \mid u_{\text{lo}}$ with $u_{\text{lo}}, u_{\text{hi}} \in [0, 65535]$, modular addition gives $(u + y) \bmod 2^{17} = \sigma_0(u) \bmod 2^{17}$, and $\mathbb{F}_2$-linearity of σ_0 gives:

$$\sigma_0(u_{\text{hi}} \ll 16) \equiv (u + y) \oplus \sigma_0(u_{\text{lo}}) \pmod{2^{17}}. \tag{E27}$$

Rather than inverting `[E27]` with a 128 KB lookup table (which incurs ~200 billion random L1 loads per sweep and contends with Q_0^{-1} loads), we prove that `[E27]` has an **exact 5-instruction register closed form**:

Theorem 2 (Closed-Form $\mathbb{F}_2$ Inverse of Upper 16 Bits). *Let $v = (u_{\text{lo}} + y) \oplus \sigma_0(u_{\text{lo}}) \pmod{2^{32}}$, and let $U = u_{\text{hi}} \ll 16 \in \mathbb{F}_2^{32}$ have bits in positions $16..31$. Define the linear operator $L(T) := T \oplus (T \ll 11) \oplus (T \ll 15) \pmod{2^{32}}$ on upper-half words T. Then L is a triangular bijection on bits $16..31$ with $U = L(T)$, where T is given in closed form by:*

$$\begin{aligned} b_0 &= \big((v \ll 15) \oplus (v \ll 11) \oplus v\big) \wedge \texttt{0x00010000}, \\ b_{12} &= \big((v \ll 14) \oplus (v \ll 3)\big) \wedge \texttt{0x00060000}, \\ T &= b_0 \mid b_{12} \mid \big((v \ll 18) \wedge \texttt{0xfff80000}\big), \end{aligned} \tag{E28}$$

and the full 32-bit preimage is recovered in 5 bitwise instructions (`reconstruct_u_dev`):

$$u = u_{\text{lo}} \mid \big(T \oplus (T \ll 11) \oplus (T \ll 15)\big). \tag{E29}$$

Proof. On bits $0..16$, $\sigma_0(U) = \mathrm{ROR}(U, 7) \oplus \mathrm{ROR}(U, 18) \oplus (U \gg 3)$ places $U_{16..31}$ at bits $9..24$, bits $30..31 \cup 0..13$, and bits $13..28$. Defining $T = L^{-1}(U)$ eliminates the two shift terms $(T \ll 11)$ and $(T \ll 15)$ inside $\mathrm{ROR}(U, 18)$, leaving a single 3-band triangular relationship between $T_{16..31}$ and $v_{0..16}$, which resolves directly into bit 16 (b_0), bits $17..18$ (b_{12}), and bits $19..31$ ($v \ll 18$). Verified exhaustively across all $2^{16} = 65{,}536$ states with 0 errors. □

6.2 Slot-Sudoku: 6 GB 2-Tier Nested 4-Slot EDF Table & 8 GB / 16 GB / 32 GB Direct Tables (`[E30]` – `[E33]`)

Because $Q_0(u) = \sigma_0(u) - u$ behaves as a random mapping on $\mathbb{Z}_{2^{32}}$, the preimage multiplicity $|Q_0^{-1}(y)|$ follows a $\mathrm{Poisson}(1)$ distribution (36.79% of y have 0 roots, 36.79% have 1, 18.39% have 2, 6.13% have 3, 1.53% have 4, and 0.37% have ≥ 5). By Theorem 2, storing only the lower 16 bits $u_{\text{lo}} = u \bmod 2^{16}$ halves the memory footprint of every table tier:

1. **6.00 GB 2-Tier Nested 4-Slot EDF + Cuckoo-Sudoku Table (`solve20_padded_fast.c`):** Designed for 16 GB laptops (3×2^{30} entries of `uint16_t` = `6.00 GB`), where every 16 consecutive y values (a **quad** $q = y \gg 4$) share **12 contiguous 16-bit slots**:

- **Earliest-Deadline-First (EDF) 3-Slot Sliding Window (`[E30]`):** At Levels 1 and 2, each y probes a 3-slot window within its 12-slot quad at offset:

$$\text{slide3_off}(y) = (y \gg 1) + (y \gg 2) - \big(\mathrm{ROR}(\texttt{0x6AAA6AAA}, y \wedge 31) \gg 31\big). \tag{E30}$$

- **Cuckoo-Sudoku Eviction & Nested 4th Slot (`[E31]`):** During construction, a cuckoo-relocation pass (`sudoku_upgrade_worker_6gb` and `tier2_nest4_quad_worker`) packs displaced preimages across each 12-slot quad, placing any 4th root at $\mathrm{b}[(y \wedge 15) \geq 13 \;?\; -1 : 3]$ (probed only when all 3 primary slots are valid roots of y). At Level 3, all 12 slots of the quad (`3 × uint64_t`) are tested simultaneously via 64-bit SWAR (`[E35]`):

$$6.00\text{ GB Coverage} = \frac{3{,}563{,}795{,}213}{2^{32}} = 82.976\%, \quad \text{3-Level Survival} = (0.82976)^3 = 57.13\%. \tag{E31}$$

2. **8.00 GB Packed 2-Way `ulo` & 16.00 GB Direct-Indexed GPU VRAM Tables (`solve20_padded_cuda.cu`):** Built natively in GPU VRAM in **`3.11 seconds`**, storing up to two 16-bit roots per y:

$$\text{2-Way Direct Coverage} = 79.64\%, \quad \text{3-Level Survival} = (0.7964)^3 = 50.51\%. \tag{E32}$$

3. **32.00 GB Direct-Indexed 4-Way Exact CPU Table (`H_exact32[y]` , `--table-gb 32`):** Stores up to four 16-bit roots per y in one `uint64_t`:

$$32.00\text{ GB Exact Coverage} = \frac{4{,}276{,}521{,}180}{2^{32}} = 99.5705\%, \quad \text{3-Level Survival} = 98.717\%. \tag{E33}$$

6.3 The 3-Level Tree Search, 64-Bit SWAR Level-3 Filter, and Terminal 20-Step Bijection (`[E34]` – `[E37]`)

For each a_1 and each Right context in a batch of size B:

- **Level 1 ($a_1 \to W_1 \to a_2$) & Level 2 ($a_2 \to W_2 \to a_3$) with Early Hamming Pruning (`[E34]`):** From a_1, look up $W_1 \in Q_0^{-1}(T_{16} + F_1(a_1))$, set $a_2 = W_1 + G_1(a_1)$, look up $W_2 \in Q_0^{-1}(T_{17} + G_2(a_1, a_2) - W_1)$, and set $a_3 = W_2 + G_2$. Before computing e_3, G_3 or issuing Level-3 DRAM loads, we prune any a_3 whose majority mask $M_3 = \neg(a_3 \oplus a_5)$ has Hamming weight exceeding $w_{\text{thresh}} \in \{13, 14\}$:

$$\text{popcount}\big(\neg(a_3 \oplus a_5)\big) \le w_{\text{thresh}}, \tag{E34}$$

  eliminating 81.8% **($w_{\text{thresh}} = 13$) to** 71.2% **($w_{\text{thresh}} = 14$)** of Level-3 lookups.

- **Level 3: 64-Bit SWAR Pre-Reconstruction Filter & 19-Step Padded Solution Stream (`[E35]`):** Because the majority condition $((a_4 \oplus a_5) \wedge M_3) = 0$ (`[E13]`) is lower-triangular on bits $0..14$, any candidate u_{lo} must satisfy $(((u_{\text{lo}} + G_3) \oplus a_5) \wedge M_3 \wedge \texttt{0x7fff}) = 0$ **before** u_{hi} is reconstructed. In the 6 GB table, we test 4 slots at a time in a single 64-bit word `w` using SWAR carry-bit detection (`K7 = 0x7fff7fff7fff7fff` , `K8 = 0x8000800080008000`):

$$d = \Big(\big(((w \wedge K_7) + g_{64}) \oplus A_{64}\big) \wedge m_{64}\Big) + K_7, \qquad \text{4-slot hit mask} = \neg d \wedge K_8, \tag{E35}$$

  rejecting $> 99\%$ of 12-slot quads in 3 register operations. Every candidate passing the full 32-bit check $((a_4 \oplus a_5) \wedge M_3) = 0$ with $Q_0(W_3) = t_{18}$ is a **complete, verified 19-step padded preimage** (`local_sol19++`).

- **Terminal 20-Step Filter & Exact $\mathbb{F}_2^{32}$ Bijection (`[E36]` – `[E37]`):** For each 19-step padded solution, we recover W_4 via `[E7]` and test the step-19 schedule equation:

$$\sigma_0(W_4) + W_3 \equiv T_{19} \pmod{2^{32}}, \qquad T_{19} := W_{19} - \sigma_1(W_{17}) - W_{12}. \tag{E36}$$

  Viewing $\sigma_0(x) = \text{ROR}(x, 7) \oplus \text{ROR}(x, 18) \oplus (x \gg 3)$ as a 32×32 matrix over $\mathbb{F}_2$, we have $\det_{\mathbb{F}_2}(\sigma_0) = 1$:

$$\det_{\mathbb{F}_2}(\sigma_0) = 1 \implies W_4 = \sigma_0^{-1}\big((T_{19} - W_3) \bmod 2^{32}\big) \text{ is uniquely determined by } W_3 \text{ and } T_{19}. \tag{E37}$$

6.4 Carry-Tree Pre-Filtering, L2 Cache-Tile Partitioning, and Octet-Hexadecet Coalescing (`[E42]` – `[E45]`)

To remove memory-bandwidth bottlenecks across Levels 1–3 on GPU and Apple Silicon unified memory (`solve20_padded_cuda.cu`), we introduce four further algorithmic optimizations (formalized in **Algorithm 2**):

1. **29-Bit Carry-Tree Pre-Filtering (`solve_tree_gpu` , `[E42]`):** Rather than sweeping all 2^{32} values of a_1 and issuing random DRAM reads at Level 1, we invert the lower-triangular structure of $F_1(a_1)$ directly in registers. For each 16 MB L2 cache tile $[\text{Tile}_{\text{lo}}, \text{Tile}_{\text{hi}})$ of the 8.00 GB 2-way `ulo` table ($K = 512$ tiles matching the 24 MB L2 cache of the NVIDIA L4), `solve_tree_gpu` constructs the exact 2^{29} subspace of a_1 roots mapping into that resident L2 tile:

$$a_1 \in F_1^{-1}\big([\text{Tile}_{\text{lo}} - T_{16},\ \text{Tile}_{\text{hi}} - T_{16})\big) \implies 87.5\% \text{ of } a_1 \text{ space pruned in registers before Level 1.} \tag{E42}$$

2. **1.0M-Context GPU Reservoir & Fused Octet-Hexadecet (T_{16}, T_{17}) Coalescing (`[E43]`):** Sorting the 2^{20}-context GPU reservoir lexicographically by $(T_{16} \gg 3,\ T_{17} \gg 3)$ forms **Octet-Hexadecet clusters ($M = 8$ pairs of $K = 16$ contexts = 128 contexts/sweep)** sharing both the same 32-byte T_{16} cache line and 8-way coalesced T_{17} sectors:

$$\Delta(T_{16} \gg 3) = 0 \ \wedge\ \Delta(T_{17} \gg 3) = 0 \quad \text{across } M = 8 \text{ context groups} \implies \mathbf{0.125}\ \text{L2 lookups per context} \tag{E43}$$

3. **Pre-`ulo2` 16-Bit Lower-Triangular Hamming Pruning (`[E44]`):** In `sweep_cycle10_integrated_kernel`, before reconstructing W_2 via `[E29]`, we exploit lower-triangularity of modular addition on bits $0..15$:

$$w_{\text{lo},2} := \text{popcount}\Big(\neg\big(((u_{\text{lo},2} + G_2) \oplus a_5) \wedge \texttt{0xffff}\big)\Big) \le w_{\text{lo,thresh}} \in \{7, 11\}, \tag{E44}$$

rejecting 65%–90% of Level-2 slots in 3 instructions and reaching $2{,}143{,}080.6\ \text{sol19/s}$ on 1× L4 GPU.

4. **The 4-Round Crossed Carry-Sudoku Spine ($a_2 = a_3 = a_4 = a_5$, `[E45]`):** Crossing the shaping constraint across 4 consecutive rounds by pinning $a_2 = a_3 = a_4 = a_5$ (`cross_round_4step_sudoku_direct_kernel`) collapses all four majority functions to a_5 and makes W_1, W_2, W_3 $O(1)$ linear functions of a_1 with zero Q_0^{-1} table lookups:

$$a_2 = a_3 = a_4 = a_5 \implies \begin{cases} \text{Maj}(a_3, a_2, a_1) = \text{Maj}(a_4, a_3, a_2) = \text{Maj}(a_5, a_4, a_3) = \text{Maj}(a_6, a_5, a_4) = a_5, \\ e_4 = a_0 - \Sigma_0(a_5), \quad W_3 = c_{W_3} - \Sigma_1(e_3) - \text{Ch}(e_3, e_2, e_1), \\ \text{5-step } (a_1 = a_5) : e_3 = a_{-1} - \Sigma_0(a_5) \implies W_3(a_0) = C_3 - (e_2 \wedge \neg e_3). \end{cases} \tag{E45}$$

Because $a_2 = a_3 = a_4 = a_5$ spends 96 Left bits, the 3 schedule equations W_{16}, W_{17}, W_{18} become a 96-bit Right filter, preserving the exact $\Theta(2^{32})$ degree-of-freedom conservation law (§9.3).

```
Algorithm 2: Cache-Tiled Left-Tree & Terminal 20-Step Preimage Solver
Input: Context cluster C of size B=128 (Octet-Hexadecet coalesced on (T_{16}>>3, T_{17}>>3)), 2-way table H_ulo
Output: Stream of 19-step padded preimages and matching 20-step padded preimage M*
1.  for each 16 MB L2 cache tile k in 0..511 do
2.    Preload tile k of H_ulo into L2 cache; invert Carry-Tree F_1^{-1} to enumerate roots a_1 in tile k  // [E42]
3.    for each root a_1 and coalesced T_{16} cache line in C do
4.      Load u_{lo,1} from resident L2 tile; reconstruct W_1 via 5-op closed form [E28]-[E29]; set a_2 = W_1 + G_1
5.      Compute coalesced T_{17} sector; load u_{lo,2} from H_ulo                                        // [E43]
6.      if popcount(~(((u_{lo,2} + G_2) ^ a_5) & 0xffff)) <= w_{lo,thresh} then                          // [E44]
7.        Reconstruct W_2 via [E28]-[E29]; set a_3 = W_2 + G_2 and M_3 = ~(a_3 ^ a_5)
8.        if popcount(M_3) <= 14 then                                                                  // [E34]
9.          Load u_{lo,3} from H_ulo[T_{18} + G_3 - W_2]; filter (((u_{lo,3}+G_3)^a_5) & M_3 & 0x7fff)==0 // [E35]
10.         Reconstruct W_3 via [E28]-[E29]; set a_4 = W_3 + G_3
11.         if ((a_4 ^ a_5) & M_3) == 0 then                                                           // [E13]
12.           Emit valid 19-step padded preimage; compute W_4 via [E7]
13.           if sigma_0(W_4) + W_3 == T_{19} (mod 2^32) then return 20-step padded preimage M*          // [E36]
```

6.5 Precomputation vs. Online Time, Memory, and SHA-256-Equivalent Complexity

Following standard cryptographic convention [2, 3, 4, 8], Table 6.1 separates **target-independent precomputation** (Q_0^{-1} table construction) from **target-dependent online complexity**, measured both in wall-clock GPU time and in **step-reduced SHA-256 compression equivalents** (`SHA-eq` , benchmarked on the same NVIDIA L4 GPU at 1.5125×10^8 SHA20-eq/s).

Table 6.1: Precomputation and online complexities of the proposed 19- and 20-step preimage attacks.

Attack & Target Setting	Precomputation Time (`SHA-eq`)	Memory Footprint	Online Context Gen (`SHA-eq`)	Online Tree Search (`SHA-eq` / Preimage)	Wall-Clock Online Time (1× L4 / 8× L4)
19-Step Padded Hash (55B / 119B)	2^{32} Q_0 evals (3.11 s)	6.0–8.0 GB	$2^{10.37}$ per ctx (8.7 ms)	$2^{6.14}$ (70.58 SHA19-eq)	0.47 μs / 0.06 μs
20-Step Padded Hash (55B / 119B)	2^{32} Q_0 evals (3.11 s)	8.0–16.0 GB	$2^{28.75}$ (9.16 s)	$2^{38.14}$ (2^{32} × 70.58 SHA20-eq)	33.4 min / 4.18 min

Attack & Target Setting	Precomputation Time (`SHA-eq`)	Memory Footprint	Online Context Gen (`SHA-eq`)	Online Tree Search (`SHA-eq` / Preimage)	Wall-Clock Online Time (1× L4 / 8× L4)
20-Step Unpadded Compression (§7)	3×2^{32} evals (45 s)	48.0 GB RAM	$O(1)$ per (a_{11}, a_{12})	$2^{36.85}$ (2^{32} sol19)	21.1 min (16 CPU cores)

7. Companion Result: 20-Step Unpadded Compression Preimages via the Zero-a_3 Bridge

When FIPS 180-4 padding is not required (the 20-step compression-function model studied in [6, 7, 9]), we can eliminate the $(3/4)^{32} \approx 2^{-13.28}$ majority survival loss of `[E13]` altogether by placing the state on the **Zero-a_3 Manifold** (`solve20_zero_a3.c`):

$$a_3 = 0, \qquad a_5 = 0, \qquad a_6 = 0, \qquad a_8 = 0, \qquad a_9 = 0, \qquad a_{10} = \texttt{0xffffffff}. \tag{E38}$$

Because $\mathrm{Maj}(0, x, 0) = 0$, both majority functions containing a_4 vanish identically for 100% **of** $a_4 \in [0, 2^{32})$:

$$\mathrm{Maj}(a_5, a_4, a_3) = \mathrm{Maj}(0, a_4, 0) = 0, \qquad \mathrm{Maj}(a_6, a_5, a_4) = \mathrm{Maj}(0, 0, a_4) = 0. \tag{E39}$$

Fixing $a_3 = 0$ removes one degree of freedom on the Left, which we restore on the Right by leaving a_7 free and solving the step-16 schedule equation via the companion operator $Q_1(u) = \sigma_1(u) + u$:

$$Q_0(W_1) = F_{\text{left}}(a_1) + R_{16}(a_7, a_{11}, a_{12}), \qquad Q_1(\Delta_{14}) = \mathrm{Target}_{Q_1}(a_1, a_{11}, a_{12}). \tag{E40}$$

Precomputing 16 GB tables for Q_0^{-1}, Q_1^{-1}, and the Left bridge (a_1, W_1, W_2) satisfying $a_3 = 0$ (48 GB total, 100% independent of H^*), `solve20_zero_a3.c` achieves **zero β-filter loss** (63.37% survival per bridge hit) and **2.16 million exact 19-step compression solutions per second** on a 16-vCPU CPU node:

$$\text{Total 20-Step Unpadded Compression Solve Time} = 1{,}266.26 \text{ s } (21.1 \text{ minutes on 16 CPU cores}). \tag{E41}$$

8. Verified 19-Step and 20-Step Padded SHA-256 Preimage Witnesses

All 29 20-step witnesses and 19-step witnesses below (covering 20-step single-block padded, 20-step two-block chosen-ASCII-prefix padded, 19-step single-block and two-block padded, and 20-step unpadded compression) are independently verified in under 1 second by running `python3 verify_all_witnesses.py` (`verify_all_witnesses.py`). Complete machine-readable witness files are provided in `demos_20step.txt` , `examples_abc_hello_20step.txt` , `witnesses_20step.txt` , and `FOUND_20ROUND_PREIMAGE.txt` .

8.1 Witness 1: First Demonstrated 20-Step Padded SHA-256 Preimage ($H^*_{20} = \texttt{0000...0000}$)

Found by `solve20_padded_cuda.cu` on Sweep 62 (`ctx=5` , after $70{,}661{,}112$ exact 19-step padded solutions; recorded in `FOUND_20ROUND_PREIMAGE.txt`):

- **Target 20-Step Digest (H^*_{20}):** `0000000000000000000000000000000000000000000000000000000000000000`
- **55-Byte Message M (440 bits, big-endian hex):** `ec531306c85ad4a4ad211d2e1a03ece4d0ad4a0e6b4c2b9436b3babdcb-d520597196f52c042b8cd95f1a39f71b58b47b0647c78e65416f`
- **FIPS 180-4 Padded 512-Bit Message Block ($W_0 \ldots W_{15}$):**

```
W[ 0.. 3] = ec531306  c85ad4a4  ad211d2e  1a03ece4
W[ 4.. 7] = d0ad4a0e  6b4c2b94  36b3babd  cbd52059
W[ 8..11] = 7196f52c  042b8cd9  5f1a39f7  1b58b47b
W[12..15] = 0647c78e  65416f80  00000000  000001b8
```

(Note the mandatory FIPS 180-4 padding [1]: $W_{13} \bmod 256 = \texttt{0x80}$*,* $W_{14} = \texttt{0x00000000}$*, and* $W_{15} = \texttt{0x000001b8} = 440$ bits*.)*

Table 8.1: Complete 20-step internal state trajectory (a_t, e_t, W_{t-1}) of the 20-step padded witness for $H^*_{20} = \texttt{0000...0000}$.

Step t	Register a_t	Register e_t	Message / Schedule Word W_{t-1}	Algebraic / Carry-Sudoku Invariant Verified
1	`e85b9b53`	`851af5a8`	$W_0 =$ `ec531306`	Swept root $a_1 \in [0, 2^{32})$
2	`47d401c6`	`4b125b48`	$W_1 =$ `c85ad4a4`	Level-1 Q_0^{-1} hit: $Q_0(W_1) = T_{16} + F_1(a_1)$
3	`0a280ce0`	`8b761050`	$W_2 =$ `ad211d2e`	Level-2 Q_0^{-1} hit; $\text{popcount}(M_3) = 10 \le 14$
4	`67298bc8`	`c121961d`	$W_3 =$ `1a03ece4`	Level-3 Q_0^{-1} hit; $((a_4 \oplus a_5) \wedge M_3) = 0$ (`[E13]`)
5	**`67978bcd`**	`2724a33e`	$W_4 =$ `d0ad4a0e`	**Exact Shaping (`[E11]`):** $a_5 = a_6 =$ `67978bcd` ($D = 0$)
6	**`67978bcd`**	`eea015fa`	$W_5 =$ `6b4c2b94`	**Exact Shaping (`[E11]`):** $a_6 = a_5 =$ `67978bcd` ($D = 0$)
7	`41b1f1b0`	`8aa086f2`	$W_6 =$ `36b3babd`	Closed-form a_7 absorbing $W_{14} = 0$ (`[E17]`)
8	`edb02749`	`9319765f`	$W_7 =$ `cbd52059`	Invariant a_8 absorbing $W_{15} = 440$ (`[E17]` , `[E22]`)
9	`7fd85be0`	**`ffffffff`**	$W_8 =$ `7196f52c`	**Selector Saturation (`[E8]`):** $e_9 =$ `0xffffffff`
10	**`a7b351d7`**	**`ffffffff`**	$W_9 =$ `042b8cd9`	**Selector Saturation (`[E8]`):** $e_{10} =$ `0xffffffff`, $a_{10} = a_{11}$
11	**`a7b351d7`**	`4f9e3855`	$W_{10} =$ `5f1a39f7`	**Majority Simplification (`[E22]`):** $a_{11} = a_{10} =$ `a7b351d7`
12	`e5d45365`	`81ba7166`	$W_{11} =$ `1b58b47b`	Right context seed a_{12}
13	`bf9f516c`	`e4acad89`	$W_{12} =$ `0647c78e`	Target-fixed a_{13}; e_{13} solves `[E21]` & `[E23]`
14	`b4322266`	`da868879`	$W_{13} =$ `65416f80`	**Padding byte (`[E21]`):** $W_{13} \bmod 256 =$ `0x80`
15	`8493b564`	**`ffffffff`**	$W_{14} =$ `00000000`	Ch-**Annihilation (`[E22]`):** $e_{15} =$ `0xffffffff`, $W_{14} = 0$
16	`97457a44`	`03b1406f`	$W_{15} =$ `000001b8`	**Padding length (`[E1]`):** $W_{15} = 440$ bits (`0x1b8`)
17..20	`5ab00ac6` .. `95f61999`	`a41f32e7` .. `aef1ad81`	$W_{16..19} =$ `d6317d0a`.. `cd1c5aee`	Terminal hit $\sigma_0(W_4) + W_3 = T_{19} \implies H_{20} =$ `00..00`

8.2 Witness 2: Flagship 20-Step & 19-Step Single-Block Padded Preimages for FIPS 180-4 Test Vector \(\text{SHA-256}(\text{`abc''})$ and $\text{SHA-256}(\text{`hello''})\)

Recorded in `demos_20step.txt` and `examples_abc_hello_20step.txt`:

- **Target Digest** $\text{SHA-256}(\text{“abc”})$ **(FIPS 180-4 §B.1 [1]):** `ba7816bf8f01cfea414140de5dae2223b00361a396177a9cb410ff61f20015ad`
- **20-Step Padded 55-Byte Preimage** $(H_{20}(M) = \text{SHA-256}(\text{“abc”}))$**:** `edf6e70a7001ae71b900f18119aaa1fdf6c1179b4268891262a9539be04d41236685e0c102e0ad23165e8b7157669819302ddcb3c68226`

```
W[ 0.. 7] = edf6e70a 7001ae71 b900f181 19aaa1fd f6c1179b 42688912 62a9539b e04d4123
W[ 8..15] = 6685e0c1 02e0ad23 165e8b71 57669819 302ddcb3 c6822680 00000000 000001b8
```

(Invariants verified: $a_5 = a_6 =$ `4ade1afa`*,* $a_{10} = a_{11} =$ `acf3c908`*,* $e_9 = e_{10} = e_{15} =$ `0xffffffff`*.)*

- **19-Step Padded 55-Byte Preimage ($H_{19}(M) = \text{SHA-256}(\text{“abc”})$):** `8ca254f8d04dd3fb04efaad6b1867588ffe833de49a144fedbf1839422892dc9f5a9158d8fc063290a14a9a6c55dfa245642a0247ec45a`
- **Target Digest $\text{SHA-256}(\text{“hello”})$:** `2cf24dba5fb0a30e26e83b2ac5b9e29e1b161e5c1fa7425e73043362938b9824`
- **20-Step Padded 55-Byte Preimage ($H_{20}(M) = \text{SHA-256}(\text{“hello”})$):** `3a11bd11262fcee01e4a184372643bdbe4ba65651b730b0b8fe49c7648866d52c60847db2400ef72c859e2b0366e470fae3cf0891313b4`
- **19-Step Padded 55-Byte Preimage ($H_{19}(M) = \text{SHA-256}(\text{“hello”})$):** `ac055aaa83e1e51a5d74102b7739b7b6e019e31100cd525baf09c642d4b59b8fc4eb8d481df111aee2616325b336b827190103c33584c5`

8.3 Witness 3: Flagship 20-Step & 19-Step Two-Block (119-Byte, Plain-ASCII First Block) Padded Preimages ($H^* = \text{SHA-256}(\text{“”})$)

Demonstrating full Merkle–Damgård multi-block chaining with an arbitrary human-readable 64-byte plain-ASCII first block (`"This first block is plain text; the second one was computed....."`) followed by a 55-byte computed second block with FIPS 180-4 padding [1] ($W_{13}^{(2)} \bmod 256 = \texttt{0x80}$, $W_{14}^{(2)} = 0$, $W_{15}^{(2)} = \texttt{0x000003b8} = 952$ bits, recorded in `demos_20step.txt`):

- **Target Digest ($H^* = \text{SHA-256}(\text{“”})$):** `e3b0c44298fc1c149afbf4c8996fb92427ae41e4649b934ca495991b7852b855`
- **Block 1 (64 Bytes Plain ASCII, identical for both 20-step and 19-step demos):** `"This first block is plain text; the second one was computed....."`

```
W^(1)[ 0.. 7] = 54686973 20666972 73742062 6c6f636b 20697320 706c6169 6e207465 78743b20
W^(1)[ 8..15] = 74686520 7365636f 6e64206f 6e652077 61732063 6f6d7075 7465642e 2e2e2e2e
```

- **20-Step Block 2 (55 Computed Bytes + 9-Byte FIPS 180-4 Padding, $L = 952$ bits $= \texttt{0x3b8}$, chaining from $H_{20}^{(1)} = \texttt{9743298d...6f586f80}$):**

```
W_20^(2)[ 0.. 7] = df28e21b b6b61202 ba87baa7 2096ee0a 3ac325cf 1f12056e 7cfe2ae1 735b4d10
W_20^(2)[ 8..15] = 6b8f6e0b 22eb9b50 4ed93689 91944b0a 115a934f 17c2c480 00000000 000003b8
```

- **19-Step Block 2 (55 Computed Bytes + 9-Byte FIPS 180-4 Padding, $L = 952$ bits $= \texttt{0x3b8}$):**

```
W_19^(2)[ 0.. 7] = fe409cc9 30128a72 0481f86c 0503d7a0 18519a12 9df44913 9720bb5e ef520895
W_19^(2)[ 8..15] = f8b51568 5bb40481 7be30188 0f0caabe 02fc9e5f 1c95c080 00000000 000003b8
```

8.4 Witness 4: Complete 20-Step and 19-Step Padded Benchmark Suites (14 Targets Each) & Unpadded Suite (13 Targets)

Matching every public-label target in the benchmark suite ($H^* = \text{SHA-256}(\text{label})$), **Table 8.2** lists the complete, untruncated 55-byte (110-hex-character) **20-step FIPS 180-4 padded preimages** (`[padded]` , where appending `8000000000000001b8` forms the 512-bit block $W_0 \ldots W_{15}$), and **Table 8.3** lists the corresponding complete 55-byte **19-step FIPS 180-4 padded preimages**.

Table 8.2: Complete 20-step single-block FIPS 180-4 padded benchmark suite (all 14 public-label targets, full 55-byte `message_hex` , $L = 440$ bits).

Target Label (`H*` = `SHA-256(label)`)	Full 256-Bit Target Digest `H*` (64 hex chars)	Complete 20-Step Padded 55-Byte Preimage `message_hex` (110 hex chars; pad suffix `8000000000000001b8`)	Verified
`"abc"`	`ba7816bf8f01cfea414140de5dae2223b00361a396177a9cb410ff61f20015ad`	`edf6e70a7001ae71b900f18119aaa1fdf6c1179b4268891262a9539be04d41236685e0c102e0ad23165e8b7157669819302ddcb3c68226`	`OK`

Target Label (`H∗ = SHA-256(label)`)	Full 256-Bit Target Digest `H∗` (64 hex chars)	Complete 20-Step Padded 55-Byte Preimage `message_hex` (110 hex chars; pad suffix `8000000000000001b8`)	Verified
`"hello"`	`2cf24dba5fb0a30e26e83b2ac5b9e29e1b161e5c1fa7425e73043362938b9824`	`3a11bd11262fcee01e4a184372643bdbe4ba65651b730b0b8fe49c7648866d52c60847db2400ef72c859e2b0366e470fae3cf0891313b4`	OK
`"padded test 1"`	`146f48dd2a3324d46feb7ad2069e05ddb9530be67f8f823f9d1c2ed3ea4f1d19`	`e2609a7b6efe43cb60db80500104189d140405a2f6b372f4e661055713c878065bc57295da59a70765407c34475c60142000cc934c6892`	OK
`"padded test 2"`	`4c51f9dde95fa3ec5545b46ca0f2d8968b2399f81f208e07304f4b00117c02df`	`b3747f89662c16d1327d8192f8dd854303bf399c7ab5dcb422fe7a287485ddc9b35c9445ac693d361ad35ee0c33f09b9e745379347451c`	OK
`"padded test 3"`	`cb9ace26d0eee1034f838de327a72598832bfa4fa1644493ad4ef004a5a2dab9`	`2a0a537e5f36b42409261161e56b241e4a816492cee5fd6897085892258dfab11ae9f922586a475b0dd2eb475d977bedaf388a94db584e`	OK
`"padded test 4"`	`90b183e68a4966c5c268807fe3f49a4f4cf890f9f25b00bb1f836552496192f4`	`312115c5023df0d90166fbb70630c77a072454beac6c71baf561dbd7644d889a339b5a1adf224599d226c509fb513fcc0e517463823afe`	OK
`"padded test 5"`	`286d96b78dc5b71f90b61e6374de3ffc82939941955e7b1c5d00fd9af345e80a`	`ac7dc4c258ea88f0d33b5adf6228fbb80f5a5e97e3f4b87452358067a178da46342609beddd43edd7b675a4c4bacfa772e3c0ed0655893`	OK
`"padded test 6"`	`efa23b479b7518cb4117dfdf041a694b46441fa5796e6d4b8a7443a724a0d8f1`	`56b98be1c35ba9d631f39ef2183cae680c4f348dbf11782f3e26fa17f5d09f480a3a03fe2ec02b3a0b5b93a42c274a830b78b19aa2b732`	OK
`"padded test 7"`	`2f507443a19f9f39300d5905b02aa1cd77c744f0424c684f75a5b2cc4a0516a8`	`40c32d88f0d6237161926436e6ff445c06cf067f56a89408e54f57482c0d109aca8bcdb09d72c5253118cd7f0e101ccc5a68e0713a6f30`	OK
`"padded test 8"`	`1fad9d7e79a132660d8536e6b151a4cef494bdfdc2962e0f7bae20acd8aed5c8`	`0b5845af8f13991a7f034a996d7cf1d4f73281307f02fde1cf59c8c86a858e24769d720c8327317e3a15f64bd78caf7cdfd85b0eb7ec12`	OK
`"padded test 9"`	`849564c85e22852e6af2c6ec60bd891de79298bbc8d5106a221bca536a908d1f`	`3917b784b65b516ef8c6a7c8b5efa09eb23d18265633d06304b438f75939ee5066bee8cf9e191247ada9185debc9c3005723e901ef6bf4`	OK
`"padded test 10"`	`91e12eb79793591d6773d8c3b5090f07a69742c7031bbd514273c5de1c241c87`	`64a7c6d93f0cf5272ef808a2fd2fa357c6e7f285ee9acb7fbafc26ae75df1f9e69231cc929e0a799b9aff6f0adfacf5f46790ff30d188b`	OK
`"padded test 11"`	`7b66f998c1c60bdd65744e43b69190481502cee718245876fc2c535f4b7cf5a8`	`95ae4eac83a9011e5566e562f14507b0dd53f8f6391a4b89718a66d9ee5a9b02366e31a8e7345cf14b70b5d82600456857a68161b2ee3b`	OK
`"hand-off cross-check 2026-09-29"`	`d374822f21eb0d4dbc809ae0443c2c6c66f8ce5d07c8779bcc5fc7849f44e3c2`	`700821b0948103e6e0ee472f6e7fac2a44139bc265ad13f422dc966e00e3c17e1f472e92b1769a7b9199a2a704fa97c927e01939f97333`	OK

Table 8.3: Complete 19-step single-block FIPS 180-4 padded benchmark suite (all 14 public-label targets, full 55-byte `message_hex`, $L = 440$ bits).

Target Label (`H∗ = SHA-256(label)`)	Complete 19-Step Padded 55-Byte Preimage `message_hex` (110 hex chars; pad suffix `8000000000000001b8`)	Verified
`"abc"`	`8ca254f8d04dd3fb04efaad6b1867588ffe833de49a144fedbf1839422892dc9f5a9158d8fc063290a14a9a6c55dfa245642a0247ec45a`	OK
`"hello"`	`ac055aaa83e1e51a5d74102b7739b7b6e019e31100cd525baf09c642d4b59b8fc4eb8d481df111aee2616325b336b827190103c33584c5`	OK
`"padded test 1"`	`a9cfb3f277e9e66e5a86ea3ff1dac69d3327b726deb9420583f150597ba70fa19cfc18400c1222c58d20911e6162081ebe084915634c3a`	OK

Target Label (`H* = SHA-256(label)`)	Complete 19-Step Padded 55-Byte Preimage `message_hex` (110 hex chars; pad suffix `8000000000000001b8`)	Verified
`"padded test 2"`	`c13089d977eea9843460eb14fff02051a5cf91b4ffedf39711c2ec81862939f8adcc0dc25fb391e5c84ed20f8ef02e4a205578ea45e9fd`	OK
`"padded test 3"`	`e24cf94c817e32304c4162886506bdfa5d2766398d36171235b6e3fde064535456052c3022ef488175bbbd6b81085c86401df4e0dae715`	OK
`"padded test 4"`	`3b605fe04bdf7b05dbcae817e1a77d8e7e9ad63db1fe8f5d972828874695dde35bcf2deb7166e78132c9ac7ee9f72d07190b83c398a2ed`	OK
`"padded test 5"`	`502eb30d876c48eaee8a13c08e4e15b5ac74689dfdf5e60cb2fd044d4bc8eaae640c92cf6236d908717a5842304f70d47fc4dddbe6d899`	OK
`"padded test 6"`	`782f45dda1ab000aace616e7c3076339fe8938a6f0a35fc6ff0e2739c3194c08494e5b1a4af0b10e69d87a02352ddd71034511dba58e90`	OK
`"padded test 7"`	`fb676bdeabfc0143be3d2b4c8350083e39b4aedcdfbf2b87c10c23786a901dc14ffc2879516522cbc86235d1a7bf135373355d0cf71888`	OK
`"padded test 8"`	`a1edbba02a9f54cf4083f7fc27156523f3acd5f52339e453c950d9ed841486dff2264d752bea2d27440984bd612c77f69b4479b63cde7f`	OK
`"padded test 9"`	`0736a2716d8ee60644b77f1e85c91f230ed82d7f242ae673baeb3049b858b159e8d90ba74901bda34b0536a82b222f425f9533e25c727c`	OK
`"padded test 10"`	`a80e3ae94cb0c81f3e591bfd4453eba61c2148ab7d32f1e524c220134d9be8e12ee2beebfa02fb6bc994272b720842d68709f1e0023b79`	OK
`"padded test 11"`	`23ae3771d98d69c6925d4ca4b8059f84404eea570db3d014c022af5649565a9a19a0e610fdb9258ce144893cee5d25f260f11614b36253`	OK
`"hand-off cross-check 2026-09-29"`	`6e77d54453883692c4e2fbf20d290ce36ee98f42748f7c618cd07a0d19b36b5a43c25fa3b008d57af11dbc2dabc843dae6117a0f867286`	OK

- **Demonstrated 20-Step Unpadded Compression Preimage (`deadbeef...` , Zero-a_3 Bridge `solve20_zero_a3.c` , plus all 12 `[comp]` targets `"unpadded test 1"` .. `"unpadded test 11"` and `"hand-off cross-check 2026-09-29"` in `witnesses_20step.txt`):**

```
W[ 0.. 7] = 4334d956 6eccece1 6727a1b2 d85811af 890c5399 79018a3b d346b94d 5f26009b
W[ 8..15] = 7562d311 0bfd80d5 1f8c38f4 16d8315d e0353aa2 6adf4bd2 a2bf14a2 801ae8f7
```

---

9. Experimental Ablation, Scope, and Structural Complexity Bounds

9.1 Algorithmic Ablation Across CPU (Apple M3) and GPU (NVIDIA L4)

Table 9.1 summarizes the algorithmic ablation of our solver from soft-shaped CSR tables to the full Carry-Sudoku / Slot-Sudoku architecture (see Table 1.1 in §1.1 for the comparison against prior reduced-step SHA-256 preimage attacks).

Table 9.1: Algorithmic ablation of 19-step and 20-step padded preimage throughput and SHA-256-equivalent cost.

Algorithmic Configuration	Q_0^{-1} Table & Memory	Right-Context Rate ($D = 0$)	19-Step Padded Rate (`sol19/s`)	`SHA20-eq` per `sol19`	Expected 20-Step Time (1× Device)	Expected Time (8× L4 GPUs)
Baseline Soft Shaping (`src/pre19.c` , 12 M3 threads)	2.0 GB CSR (2^{29} slots, $D \le 3$)	Soft ($D \le 3$)	~ 200 sol19/s	$\sim$ 120,000	~ 7.6 days	—
Exact Shaping + Slot-Sudoku CPU (`solve20_padded_fast.c`)	6.0 GB 2-Tier 4-Slot EDF (`[E30]` – `[E31]`)	9.1 ctx/s (86.4B/s)	6,242.8 sol19/s	11,400	11.9 hours	—

Algorithmic Configuration	Q_0^{-1} Table & Memory	Right-Context Rate ($D = 0$)	19-Step Padded Rate (`sol19/s`)	`SHA20-eq` per `sol19`	Expected 20-Step Time (1× Device)	Expected Time (8× L4 GPUs)
Slot-Sudoku + 5-Op $\mathbb{F}_2$ Inverse GPU (`[E29]` , 1× L4)	6.0 GB 2-Tier EDF ($B = 8$ sector)	$7{,}580$ ctx/s	$73{,}560.5$ sol19/s	$6{,}950$	16.22 hours	2.03 hours
Direct 2-Way + Warp L3 GPU (`[E32]` , 1× L4)	16.0 GB Direct 2-Way (16-way T_{16})	$29{,}310$ ctx/s	$180{,}831.9$ sol19/s	$2{,}828$	6.60 hours	49.5 min
29-Bit Carry-Tree + L2 Tiling (`[E42]` , 1× L4)	8.0 GB 2-Way `ulo` ($K = 512$ L2 tiles)	$29{,}310$ ctx/s	$759{,}664.5$ sol19/s	673.2	1.57 hours	11.8 min
Octet-Hexadecet + Pre-`ulo2` Pruning (`[E43]` – `[E44]` , 1× L4)	**8.0 GB `ulo` (** 0.125 L2/ctx**)**	$114{,}507$ ctx/s	$2{,}143{,}080.6$ sol19/s	70.58 ($2^{6.14}$)	33.4 min ($2^{38.14}$ SHA20-eq)	4.18 min

9.2 Scope, Terminology, and Limitations: Subspace-Restricted Preimages, Degree-of-Freedom Budget, Exact-T_{16} Twin Capacity, and the 15-Theory Structural Barrier Theorem

To situate our results accurately within the SHA-256 cryptanalysis literature, we distinguish three notions of restriction in reduced-step preimage attacks:

1. **Unpadded Compression Function vs. Full FIPS 180-4 Padded Hash Function:** Prior practical 18-, 19-, and 20-step results [6, 7, 9] explicitly restrict their claims to the **unpadded compression function** ($W_0, \ldots, W_{15} \in \{0,1\}^{512}$ unconstrained). Zaikin [7] specifies that Merkle–Damgård padding is omitted while keeping the standard IV and feed-forward, and Basra [9] notes: *"These results concern reduced-round compression, not padded or full SHA-256."* By contrast, our main 19- and 20-step solvers (`solve20_padded_fast.c` , `solve20_padded_cuda.cu` , `solve20_padded_cuda_prod.cu`) satisfy all 72 bits of single-block FIPS 180-4 padding [1] ($W_{13} \bmod 256 = \texttt{0x80}$, $W_{14} = 0$, $W_{15} = 440$), producing valid 55-byte (and 119-byte two-block) messages for the step-reduced hash function itself.

2. **Subspace-Restricted Preimages, Exact-T_{16} Twin Capacity, and the Degree-of-Freedom Budget:** Like all splice-and-cut MitM and biclique attacks on padded SHA-256 [2, 3, 4, 8, 10], which fix message lengths in (W_{14}, W_{15}) and pin internal state registers ("neutral words" / "initial structures") to decouple forward and backward chunks, our solver restricts its search to a structured **55-byte single-block internal-state manifold** defined by `[E8]` , `[E11]` , `[E13]` , and `[E22]` . Because a 55-byte message contains 440 free message bits against a 256-bit digest, the unconstrained message space holds $2^{440-256} = 2^{184}$ expected 20-step preimages per target H^*. Table 9.3 tracks how pinning internal state registers trades this surplus of solutions for $O(1)$ decoupling without exhausting the solution space.

Table 9.3: Degree-of-freedom budget across the 55-byte single-block padded manifold.

Constraint / Algebraic Pinning	Equation	Bits Spent	Remaining 19-Step Padded Solutions	Remaining 20-Step Padded Solutions
Full 55-byte padded message space (W_{13} pad, $W_{14} = 0, W_{15} = 440$)	`[E1]`	72 bits (of 512)	$2^{216.0}$	$2^{184.0}$
Selector saturation ($e_9 = e_{10} = \texttt{0xffffffff}$) [9]	`[E8]`	64.0 bits	$2^{152.0}$	$2^{120.0}$
Exact context shaping ($a_5 = a_6 \iff D_{\text{shape}} = 0$)	`[E11]`	32.0 bits	$2^{120.0}$	$2^{88.0}$
Majority absorption ($\mathrm{Maj}(a_5, a_4, a_3) = a_4$ on $w \leq 14$)	`[E13]`	≈ 9.2 bits	$2^{110.8}$	$2^{78.8}$
Right-context Ch-annihilation ($e_{15} = \texttt{0xffffffff}$, $a_{10} = a_{11}$)	`[E22]`	64.0 bits	$2^{40.94}$ ($\approx 2^{39.95}$ in 2-way table)	$2^{8.94} \approx 490$–805 (≈ 805 in 2-way table)

- **Carry-Sudoku Context Universe ($U \approx 1.014 \times 10^7$) vs. Exact-$T_{16}$ Twin Capacity (`single4` Mode):** Across all 2^{32} values of a_{12} for a given target H^*, Algorithm 1 (`[E22]`–`[E26]`) produces an exact universe of $U \approx 1.014 \times 10^7 \approx 2^{23.27}$ valid right-hand contexts. Since each context yields $\approx 1.06 \times 10^5 \approx 2^{16.68}$ exact 19-step padded solutions in the 2-way table across $a_1 \in [0, 2^{32})$, the full universe U yields $2^{23.27} \times 2^{16.68} \approx 805 \times 2^{32}$ 19-step solutions—i.e., ≈ 805 **distinct 20-step padded preimages per target** ($\approx 1{,}590$ with the 32 GB 4-way table). Pairing contexts by exact 32-bit birthday collisions $T_{16}^{(i)} = T_{16}^{(j)}$ yields $N_{\text{twins}} \approx \frac{U^2}{2 \cdot 2^{32}} \approx 11{,}970$ disjoint pairs (`187` sweeps of $B = 128$, $\approx 1.90 \times 2^{32}$ 19-step solutions), which suffices to solve a single target within the twin subset with Poisson probability $1 - e^{-1.90} = 85.0\%$ (empirically $11/14 = 78.6\%$ of single-block targets plus the 2-block `""` demo). For the remaining 15% of targets (`"abc"`, `"padded test 6"`, `"padded test 9"`), sampling uniformly from the full $U \approx 1.014 \times 10^7$ reservoir using 4-way register-prefetched Level-1 lookups (`single4`, `solve20_padded_cuda_prod.cu`) solves all remaining targets with 100% success across all 16 padded targets.
- **Binary 55-Byte Output Format vs. Two-Block Chosen-ASCII Prefix (119 Bytes):** In a single 55-byte block, $W_0, \ldots, W_{10}$ and the upper 24 bits of W_{13} are determined algebraically by the state trajectory. As demonstrated in §8.3, a **119-byte two-block message** ($W_{15}^{(2)} = 952 = \texttt{0x3b8}$) allows the first 64-byte block B_1 to be arbitrary ASCII text while Block 2 is solved from chaining state $\text{IV}' = \text{Compress}_{20}(\text{IV}, B_1)$ at identical complexity.

3. **The 15-Theory Structural Barrier Theorem ($O(1)$ at 19 Steps, $\Theta(2^{32})$ at 20 Steps, $\Theta(2^{64})$ at 21 Steps):** To determine whether *any* algebraic, differential, polynomial, cross-round, or multi-block shortcut can reduce 20-step preimages below $\Theta(2^{32})$, we formulated and tested **15 candidate sub-2^{32} theories** (`test_all_12_theories.c`, `discover_r20_signals.c`, Table 9.4).

Table 9.4: Summary of the 15 algebraic, statistical, and multi-block theories tested against the $\Theta(2^{32})$ 20-step barrier.

#	Candidate Sub-2^{32} Theory Tested	Mathematical Mechanism	Empirical / Algebraic Result	Complexity Verdict
1	Rightward Spine Shift ($e_{10} = e_{11} = \texttt{0xffffffff}$)	Shift bridge wall from a_5 to a_6 to gain 5 Left variables	W_{16} requires $W_9(a_5, e_9)$ at Level 1 before a_5 is known (`W16 <-> W9` Causality Law)	Circular dependency ($\geq 2^{32}$)
2	Exact $\mathbb{F}_2^{32}$ Terminal Inversion (`[E37]`)	Invert $\sigma_0(W_4) = T_{19} - W_3$ directly via $\det_{\mathbb{F}_2}(\sigma_0) = 1$	Uniquely locks W_4, which overdetermines a_4 against Level-3 $Q_0(W_3) = t_{18}$	32-bit filter ($\Theta(2^{32})$)
3	Backward 4-Level Tree ($W_4 \to W_3 \to W_2 \to W_1$)	Start from T_{19} and walk backward to a_1	W_4 requires (e_4, e_3, e_2, e_1), which depend non-triangularly on (a_4, a_3, a_2, a_1)	Non-causal ($\geq 2^{32}$)
4	Cross-Context Meet-in-the-Middle (T_{19} Multiplexing)	Reuse one Left chain $(a_1..a_4)$ across M Right contexts	Left chain depends on the 96-bit triple (T_{16}, T_{17}, T_{18}); collision requires 2^{48} contexts	Birthday wall ($2^{48} \gg 2^{32}$)
5	LSB Carry-Chain Early Pruning on $\sigma_0(W_4) + W_3$	Prune a_4 bit-by-bit using LSBs of T_{19}	$\sigma_0(W_4)$ mixes MSB bits of W_4 into bit 0 via $\text{ROR}(W_4, 7) \oplus \text{ROR}(W_4, 18)$	Full 32-bit mixing
6	Differential / Boomerang Trail on $a_1 \mapsto \sigma_0(W_4) + W_3$	Flip bit i of a_1 to induce a predictable ΔT_{19}	Measured across all 32 bit flips: exact **16.000 / 32 bits** avalanche (perfect PRF)	Zero differential bias
7	Linear / Walsh-Hadamard Approximation of $\sigma_0(W_4) + W_3$	Correlate linear masks $\alpha \cdot a_1 \oplus \beta \cdot T_{19}$	Max correlation across 3-level Q_0^{-1} composition is indistinguishable from 2^{-16} noise	Zero linear bias
8	$\mathbb{F}_2$ / ANF Polynomial Degree Reduction (Gröbner / SAT)	Express $T_{19}(a_1)$ as low-degree boolean polynomials	3-level modular addition + Q_0^{-1} reaches algebraic degree 31 on all 32 output bits	Full degree 31 ($\Theta(2^{32})$)
9	Invariant Subspace / Fixed-Point Manifold of Q_0^{-1}	Restrict W_1, W_2, W_3 to σ_0-invariant subspaces	Affine shifts T_{16}, T_{17}, T_{18} destroy $\mathbb{F}_2$-subspace invariance at every level	$\Theta(2^{32})$
10	2-Way Bi-Directional Bridge inside Single Block ($a_1 \leftrightarrow a_4$)	Meet at a_2 or a_3 from both ends of Left chain	e_3 and e_4 couple (a_4, a_3, a_2, a_1) across 4 steps simultaneously; memory $\geq 2^{32}$	$\Theta(2^{32})$ time-memory
11	Soft-Shaping Relaxation ($D_{\text{shape}} = a_5 \oplus a_6 \neq 0$)	Allow $D_{\text{shape}} \neq 0$ to let a_4 steer W_{11}, W_{12} into T_{19}	Every non-zero bit in D_{shape} halves Level-3 survival, canceling any T_{19} freedom	Net gain $\leq 1\times$

#	Candidate Sub-2^{32} Theory Tested	Mathematical Mechanism	Empirical / Algebraic Result	Complexity Verdict
12	Unfrozen $e_{15} \neq$ `0xffffffff` / $a_{10} \neq a_{11}$ Right Coupling	Vary e_{15} or $a_{10} - a_{11}$ so multiple contexts share (T_{16}, T_{17}, T_{18})	Unlocks $2^{78.8}$ total solutions, but each context still has independent 96-bit (T_{16}, T_{17}, T_{18})	$\Theta(2^{32})$ per solution
13	**4-Round Crossed Carry-Sudoku Spine ($a_2 = a_3 = a_4 = a_5$, `[E45]`)**	Pin $a_2 = a_3 = a_4 = a_5$ so W_1, W_2, W_3, W_4 are $O(1)$ with 0 Q_0^{-1} lookups	Verified at $9{,}617.2\text{M evals/s}$ on L4 GPU, but spends 96 Left bits, turning T_{16}, T_{17}, T_{18} into a 96-bit Right filter	Conservation of DoF ($\Theta(2^{32})$)
14	**5-Step Crossed Spine ($a_1 = a_2 = a_3 = a_4 = a_5$, `[E45]`)**	Pin all 5 Left words so $e_3 = a_{-1} - \Sigma_0(a_5)$	Verified exact on GPU; spends all 128 Left bits, requiring 2^{128} Right contexts	Conservation of DoF
15	**Two-Block Feed-Forward Coupling ($M = B_1 \parallel B_2$, 119 Bytes)**	Use free 512-bit Block 1 (B_1) to steer chaining state $V = H^{(1)}$ into Block 2	By Davies–Meyer feed-forward, $H^*_{20} = V + S^{(2)}_{20} \pmod{2^{32}}$: fixing $S^{(2)}_{20}$ uniquely locks all 256 bits of $V = H^*_{20} - S^{(2)}_{20}$	**Two-Block Locking ($\Theta(2^{32})$)**

- **Proof of Theory 15 (Two-Block Feed-Forward Locking):** In a two-block 119-byte padded message $M = B_1 \parallel B_2$, Block 1 ($B_1 \in \{0,1\}^{512}$) maps $\text{IV} \mapsto V = \text{Compress}_{20}(\text{IV}, B_1)$, and Block 2 maps $V \mapsto S^{(2)}_{20}$ with Davies–Meyer feed-forward $H^*_{20} = V + S^{(2)}_{20} \pmod{2^{32}} \iff S^{(2)}_{20} = H^*_{20} - V \pmod{2^{32}}$. Because V enters Block 2 at **both** boundaries simultaneously (Step 0 as V and Step 20 as $H^*_{20} - V$), choosing B_1 first fixes both boundaries and reduces Block 2 to a single-block 20-step padded problem ($\Theta(2^{32})$), while choosing Block 2's Right context first uniquely locks all 256 bits of $V = H^*_{20} - S^{(2)}_{20}$, requiring a full 256-bit 20-step compression preimage on Block 1 ($\Theta(2^{32})$). Meeting in the middle on the 256-bit chaining value V requires $2^{128} \gg 2^{32}$.
- **Resulting Complexity Ladder:** Across both single-block and multi-block architectures under realistic memory, each step beyond 19 imposes an exact 2^{32} filter while spending 32 bits of the remaining 78.8-bit solution surplus: **19 steps is** $O(1)$ ($2^{6.14}$ SHA19-eq), **20 steps is** $\Theta(2^{32})$ ($2^{38.14}$ SHA20-eq; $2^{78.8}$ solutions exist), **21 steps is** $\Theta(2^{64})$ ($2^{46.8}$ solutions exist), **22 steps is** $\Theta(2^{96})$ ($2^{14.8}$ solutions exist), and **23 steps has** 0 **solutions** inside the $e_9 = e_{10} =$ `0xffffffff`, $a_5 = a_6$ manifold.

Disclosure & Attribution

The selector-saturation and context-shaping principle ($e_9 = e_{10} =$ `0xffffffff`, $a_5 \approx a_6$) is due to K. S. Basra [9] (`https://github.com/kamb-code/sha256-r19-preimage`), whose manuscripts note substantial assistance from AI tools. Both `srw/sha256-19step-preimages-with-padding` and the Carry-Sudoku / Slot-Sudoku solvers and witnesses in this work were developed with AI assistance.

References

1. **[1]** National Institute of Standards and Technology: *Secure Hash Standard (SHS)*. FIPS PUB 180-4, U.S. Department of Commerce (2015).
2. **[2]** Isobe, T., Shibutani, K.: *Preimage Attacks on Reduced Tiger and SHA-2*. In: Dunkelman, O. (ed.) Fast Software Encryption – FSE 2009, LNCS, vol. 5665, pp. 139–155. Springer (2009).
3. **[3]** Aoki, K., Guo, J., Matusiewicz, K., Sasaki, Y., Wang, L.: *Preimages for Step-Reduced SHA-2*. In: Matsui, M. (ed.) Advances in Cryptology – ASIACRYPT 2009, LNCS, vol. 5912, pp. 578–597. Springer (2009).
4. **[4]** Khovratovich, D., Rechberger, C., Savelieva, A.: *Bicliques for Preimages: Attacks on Skein-512 and the SHA-2 Family*. In: Canteaut, A. (ed.) Fast Software Encryption – FSE 2012, LNCS, vol. 7549, pp. 244–263. Springer (2012).
5. **[5]** Homsirikamol, E., Morawiecki, P., Rogawski, M., Srebrny, M.: *Security Margin Evaluation of SHA-256 through SAT-Based Cryptanalysis*. In: CISIM 2012, LNCS, vol. 7564, pp. 56–67. Springer (2012); IACR ePrint 2012/421.
6. **[6]** Davydov, A., Pikhtovnikov, A., Kiryanova, E., Zaikin, O.: *Cryptographic Strength Analysis of the SHA-256 Hash Function Using the SAT Approach*. Scientific and Technical Journal of Information Technologies, Mechanics and Optics (2025).

7. **[7]** Zaikin, O.: *Inverting 19-Round SHA-256 Compression Functions via Cube-and-Conquer*. Constraints (Jan 2026).
8. **[8]** Guo, J., Li, M., Liu, G., Wang, M., Zhang, L.: *Improved Meet-in-the-Middle Preimage Attacks on Step-Reduced SHA-256/512*. In: Advances in Cryptology – EUROCRYPT 2026. Springer (2026); IACR ePrint 2026/353.
9. **[9]** Basra, K. S.: *Context Shaping for 19- and 20-Round SHA-256 Compression Preimages*. GitHub manuscript, `https://github.com/kamb-code/sha256-r19-preimage` (Sep 2026).
10. **[10]** Sasaki, Y., Aoki, K.: *Finding Preimages in Full MD5 Faster Than Exhaustive Search*. In: Joux, A. (ed.) Advances in Cryptology – EUROCRYPT 2009, LNCS, vol. 5479, pp. 134–152. Springer (2009).
11. **[11]** Mendel, F., Nad, T., Schläffer, M.: *Improving Local Collisions: New Attacks on Reduced SHA-256*. In: Johansson, T., Nguyen, P. Q. (eds.) Advances in Cryptology – EUROCRYPT 2013, LNCS, vol. 7881, pp. 262–278. Springer (2013).
12. **[12]** Mendel, F., Nad, T., Schläffer, M.: *Finding SHA-2 Characteristics: Searching through a Minefield of Contradictions*. In: Lee, D. H., Wang, X. (eds.) Advances in Cryptology – ASIACRYPT 2011, LNCS, vol. 7073, pp. 288–307. Springer (2011).
13. **[13]** De Cannière, C., Rechberger, C.: *Finding SHA-1 Characteristics: General Results and Applications*. In: Lai, X., Chen, K. (eds.) Advances in Cryptology – ASIACRYPT 2006, LNCS, vol. 4284, pp. 1–20. Springer (2006).
14. **[14]** Li, H., Isobe, T., Shibutani, K.: *New Practical/Semi-Free-Start Collision Attacks on Reduced SHA-256*. IACR Transactions on Symmetric Cryptology (ToSC), 2024(1), pp. 131–162 (2024).
15. **[15]** Li, M., Liu, G., Qin, L., Sun, B., Wang, M.: *MILP-Based Automatic Search for Meet-in-the-Middle Preimage Attacks on SHA-2*. IACR Transactions on Symmetric Cryptology (ToSC), 2023(4), pp. 210–241 (2023).
16. **[16]** Dobraunig, C., Eichlseder, M., Mendel, F.: *Analysis of SHA-512/224 and SHA-512/256*. In: Iwata, T., Cheon, J. H. (eds.) Advances in Cryptology – ASIACRYPT 2015, LNCS, vol. 9453, pp. 612–630. Springer (2015).
17. **[17]** McDair: *17-Step Reduced SHA-256 Preimage Challenge Witnesses*. Cryptography Mailing List (`metzdowd.com`), Mar 2024.
18. **[18]** Stevens, M., Bursztein, E., Karpman, P., Albertini, A., Markov, Y.: *The First Collision for Full SHA-1*. In: Katz, J., Shacham, H. (eds.) Advances in Cryptology – CRYPTO 2017, LNCS, vol. 10401, pp. 570–596. Springer (2017).
19. **[19]** Leurent, G., Peyrin, T.: *SHA-1 is a Shambles: First Chosen-Prefix Collision on SHA-1 and Application to the PGP Web of Trust*. In: 29th USENIX Security Symposium (USENIX Security 2020), pp. 1839–1856 (2020).
20. **[20]** Nossum, V.: *SAT-Based Preimage Attacks on SHA-1*. Master's Thesis, University of Oslo (2012).